Velocity_Verlet_Integrator_UPDATED

October 8, 2026

1 3D Velocity-Verlet Symplectic Integrator (all calculations done in a 3D Cartesian Coordinate System) with standard plummer softening implementation for correct energy conservation.

2 Uses the Velocity-Verlet algorithm to validate kepler's law and run some numerical solutions to the 3-body problem.

3 Must have the correct parameter files within the same directory as this notebook in order to run.

4 Includes parallelisation for the Kepler's 3rd Law validation runs to speed things up (spawning n-cores number of simulations at once with varying distances between two bodies to validate Kepler's 3rd Law).

5 Imports and configuration

[1]:
```
### Imports ###

# Array calculations
import numpy as np

# Plotting
import matplotlib.pyplot as plt
from IPython.display import display

# Simulation progress indicator
from tqdm import tqdm

# File reading
import os

# Multiprocessing
from multiprocessing import Pool
```

```
# Warnings for period guessing
import warnings
```

[2]:
```
### Configuring packages ###

# filtering out any 64-bit machine precision residuals that crop up in the␣
 ↪calculations in numpy
np.set_printoptions(suppress=True, precision=10)

# Configuring matplotlib display for inline plots
%matplotlib inline
```

6 Sanity check: Checking that all CPU cores are available for multiprocessing

[3]:
```
# Defining a global CPU core count
n_cores = os.cpu_count()

if type(n_cores) != int: # Must have an integer number of cores in any CPU

    raise ValueError('Critical error with python environment: os package has␣
 ↪not registered all the CPU cores, please check os is working correctly.')

else:
    print('All CPU cores detected successfully.')
    print(f'Number of available CPU cores for multiprocessing is {n_cores}.')
```

```
All CPU cores detected successfully.
Number of available CPU cores for multiprocessing is 24.
```

7 Sanity Check: Checking the multiprocessing files are present

[4]:
```
# Checking that all the parallelisable worker files exist
if os.path.exists('parallelisable_integrator_functions.py') != True:

    raise ValueError('Critical error with multiprocessing files: The files␣
 ↪required for multiprocessing are either damaged or not in the same directory␣
 ↪as this code.\nPlease ensure␣
 ↪parallelised_lattice_side_length_investigation_functions_MMC.py and␣
 ↪parallelised_temperature_investigation_functions_MMC.py are both in the same␣
 ↪directory as this notebook before restarting the notebook kernel and␣
 ↪re-running.')

else:
```

```
    print('All multiprocessing files successfully detected. Importing␣
↪parallelisable functions and proceeding with runs.')

    # Importing the multiprocessing file
    from parallelisable_integrator_functions import run_integrator_worker
```

```
All multiprocessing files successfully detected. Importing parallelisable
functions and proceeding with runs.
```

8 Function to read in 2 body file, includes safety check in case file is not there

[5]:
```
# Read 2 body ics
def read_two_body_file(parameter_file):

    """Reads in the parameter file after checking it is there, and returns the␣
↪parameters in the following formats:

    masses, initial_positions, initial_velocities: numpy arrays.
    G, soft, timestep, period_tol: float.
    nsteps: int
    filename: string.

    Ignores any lines starting with a #

    Returns the arguements in the same sequence as the␣
↪validate_kep_two_body_system() function takes them:
    mass_1, mass_2, dist_min, dist_max, num_tests, G, soft, period_tol,␣
↪system_type, mode."""

    # Checking if the file is in the same directory as this notebook
    if os.path.exists(parameter_file) is False:

        raise ValueError('File is not in the same directory as this notebook.␣
↪Please move the file to the same directory as the notebook and re-run')

    # Empty dictionary to store the parameters
    parameter_storage_dict = {}

    with open(parameter_file, 'r') as f:

        for txtline in f:

            # Skipping empty lines and lines starting with a hashtag
            txtline = txtline.strip() # Removing spaces - Makes things easier
            if not txtline or txtline.startswith('#'):
```

```
                continue

            # Detecting assigned parameters in the file
            if '=' in txtline:
                key, parameter = txtline.split('=', 1) # Split the line based␣
↪on the first occurrence of = character
                key = key.strip()
                value = parameter.strip()

                # Assigning the parameter
                parameter_storage_dict[key] = parameter

    # Sanity checks for dictionary
    for key, parameter in zip(parameter_storage_dict.keys(),␣
↪parameter_storage_dict.values()):

        if key == None or parameter == None:
            raise ValueError('Parameter file is faulty: Conatains variables␣
↪that point to no values or values with no assigned variable.')

        else:
            continue

    # Extracting simulation numerical parameters
    G = float(parameter_storage_dict['G'])
    soft = float(parameter_storage_dict['soft'])
    period_tol = float(parameter_storage_dict['period_tol'])
    num_tests = int(parameter_storage_dict['num_tests'])

    # Extracting masses
    mass_1 = float(parameter_storage_dict['mass_1'])
    mass_2 = float(parameter_storage_dict['mass_2'])

    # Extracting distances
    dist_min = float(parameter_storage_dict['dist_min'])
    dist_max = float(parameter_storage_dict['dist_max'])

    # Extracting system configuration
    system_type = parameter_storage_dict.get('system_type')
    system_type = system_type.strip() # Eliminating rogue whitespaces
    mode = parameter_storage_dict.get('mode')
    mode = mode.strip() # Eliminating rogue whitespaces

    # Returning parameters in the same sequence as the run_integrator function␣
↪takes them
```

```
    return mass_1, mass_2, dist_min, dist_max, num_tests, G, soft, period_tol,␣
↪system_type, mode
```

9 Function to read in 3 body file, includes saftey check in case file is not there

[6]:
```
# Read 3 body ics
def read_three_body_file(parameter_file):

    """Reads in the parameter file after checking it is there, and returns the␣
↪parameters in the following formats:

    masses, initial_positions, initial_velocities: numpy arrays.
    G, soft, timestep, period_tol: float.
    nsteps: int
    filename: string.

    Ignores any lines starting with a #

    Returns the arguements in the same sequence as the run_integrator function␣
↪takes them:
    masses, initial_positions, initial_velocities, G, soft, timestep, nsteps,␣
↪period_tol, filename."""

    # Checking if the file is in the same directory as this notebook
    if os.path.exists(parameter_file) is False:

        raise ValueError('File is not in the same directory as this notebook.␣
↪Please move the file to the same directory as the notebook and re-run')

    # Empty dictionary to store the parameters
    parameter_storage_dict = {}

    with open(parameter_file, 'r') as f:

        for txtline in f:

            # Skipping empty lines and lines starting with a hashtag
            txtline = txtline.strip() # Removing spaces - Makes things easier
            if not txtline or txtline.startswith('#'):

                continue

            # Detecting assigned parameters in the file
            if '=' in txtline:
```

```
                key, parameter = txtline.split('=', 1) # Split the line based␣
↪on the first occurrence of = character
                key = key.strip()
                value = parameter.strip()

                # Assigning the parameter
                parameter_storage_dict[key] = parameter

    # Sanity checks for dictionary
    for key, parameter in zip(parameter_storage_dict.keys(),␣
↪parameter_storage_dict.values()):

        if key == None or parameter == None:
            raise ValueError('Parameter file is faulty: Conatains variables␣
↪that point to no values or values with no assigned variable.')

        else:
            continue

    # Extracting parameters to return if the file contains no unassigned␣
↪variables
    # Extracting filename
    filename = parameter_storage_dict.get('filename')
    filename = filename.strip() # Important to remove spaces - OneDrive has␣
↪backup issues if the filename has spaces.

    # # Extracting simulation numerical parameters
    G = float(parameter_storage_dict['G'])
    soft = float(parameter_storage_dict['soft'])
    timestep = float(parameter_storage_dict['timestep'])
    period_tol = float(parameter_storage_dict['period_tol'])
    nsteps = int(parameter_storage_dict['nsteps'])

    # Creating the numpy arrays
    # Extracting masses
    masses = np.array([float(mass) for mass in parameter_storage_dict['masses'].
↪split(',')], dtype='float64')

    # Extracting positions
    # Splitting based on comma separation
    position_m1 = [float(coord_m1) for coord_m1 in␣
↪parameter_storage_dict['position_m1'].split(',')]
    position_m2 = [float(coord_m2) for coord_m2 in␣
↪parameter_storage_dict['position_m2'].split(',')]
    position_m3 = [float(coord_m3) for coord_m3 in␣
↪parameter_storage_dict['position_m3'].split(',')]
```

```
    # Array of positions
    initial_positions = np.array([position_m1, position_m2, position_m3],␣
↪dtype='float64')

    # Extracting velocities
    # Splitting the extracted string based on comma separation
    velocity_m1 = [float(velocity_m1) for velocity_m1 in␣
↪parameter_storage_dict['velocity_m1'].split(',')]
    velocity_m2 = [float(velocity_m2) for velocity_m2 in␣
↪parameter_storage_dict['velocity_m2'].split(',')]
    velocity_m3 = [float(velocity_m3) for velocity_m3 in␣
↪parameter_storage_dict['velocity_m3'].split(',')]

    # Array of velocities
    initial_velocities = np.array([velocity_m1, velocity_m2, velocity_m3],␣
↪dtype='float64')

    # Returning parameters in the same sequence as the run_integrator function␣
↪takes them
    return masses, initial_positions, initial_velocities, G, soft, timestep,␣
↪nsteps, period_tol, filename
```

10 Function to compute acceleration due to gravity

[7]:
```
# Calculating acceleration using newton's law of gravitation and the positions␣
↪of the bodies only
def compute_acceleration(masses, positions, G, soft):

    """Computes the acceleration of each body using newton's law of gravitation␣
↪and the positions and masses of the other bodies,
        which is extracted from the initial conditions before the integration␣
↪begins

        The bodies are indexed twice in order to check that the body's␣
↪acceleration is not computed using its own position, which
        would cause the simulation to break down due to dividing by zero."""

    # Array used to store the acceleration due to gravity of each body
    gravitational_acceleration = np.zeros(positions.shape, dtype='float64')

    # Getting the number of bodies from the number of masses.
    num_bodies = len(masses)

    # Indexing to extract array and position elements from the array.
```

```
    for i in range(num_bodies):
        for j in range(num_bodies):

            # Double indexing to prevent acceleration being calculated for the␣
↪self-position of a body which means dividing by zero
            # Continiue means do nothing in this context
            if i == j:

                continue

            # Distances between each body, with no self-positions due to double␣
↪indexing
            # Plummer softening from Alexander Shirokov 2008 https://arxiv.org/
↪pdf/0711.2989 implemented (see equation 1 in the paper)
            distance = positions[j] - positions[i]
            distance_squared = (np.linalg.norm(distance))**2

            # Acceleration of each body computed with the positions of the␣
↪other bodies
            gravitational_acceleration[i] += (G * masses[j] * distance) /␣
↪(distance_squared + soft**2)**1.5

    return gravitational_acceleration
```

11 Function to perform a Verlet integration step

[8]:
```
# Kick drift kick form of verlet integration.
def verlet_integration(masses, old_positions, old_velocities, old_acceleration,␣
↪G, soft, timestep):

    """Solves differential equations of motion using a velocity verlet method.␣
↪Velocity is dependent only on acceleration, and
    unlike leapfrog integration, the velocity is explicitly calculated and␣
↪handled as a variable."""

    # New position computed from initial coniditions only
    position_timestep = old_positions + old_velocities * timestep + 0.5 *␣
↪old_acceleration * timestep**2

    # Recalculating accelerations
    new_acceleration = compute_acceleration(masses, position_timestep, G, soft)

    # Calculate velocity from the new acceleration and the positions
    velocity_timestep = old_velocities + 0.5 * (old_acceleration +␣
↪new_acceleration) * timestep
```

```
    return position_timestep, velocity_timestep, new_acceleration
```

12 Function to calculate angular momentum

[9]:
```
# Calculating total angular momentum using a vectorised method
def calculate_angular_momentum(masses, positions_storage_array,␣
↪velocities_storage_array):

    """Calculates the angular momentum of the entire system by performing the␣
↪cross product of the positions with the momentum vector
    of each body. This function is designed to be run once per integration loop.
↪ The positions are calculated with respect
    to a common origin. Takes arguement as the arrays generated by the␣
↪run_integrator function"""

    # Calculating linear momentum
    linear_momentum_array = masses[:, np.newaxis] * velocities_storage_array

    # Calculating angular momentum
    angular_momentum_array = np.cross(positions_storage_array,␣
↪linear_momentum_array)

    # Summing up the angular momentum of each particle for each timestep
    # This yields the system angular momentum at each timestep
    total_system_angular_momentum = np.sum(angular_momentum_array, axis=0)

    # Returning the magnitude of the angular momentum so it can be plotted wrt␣
↪to timestep
    return np.linalg.norm(total_system_angular_momentum)
```

13 Function to calculate the total system energy (potential + kinetic)

[10]:
```
# Calculating the total energy (potential + kinetic)
def calculate_total_energy(masses, positions_storage_array,␣
↪velocities_storage_array, G, soft):

    """Calculates the total energy in the system by calculating the kinetic and␣
↪potential enegry separately and adding them
    together at the end using a singular nested for loop, with kinetic energy␣
↪being on the outer loop because
    it does not depend on distances unlike GPE. Takes arguements from the """
```

```
    # Getting number of bodies
    num_bodies = len(masses)

    # Initialising kinetic energy to start with - will sum over later in the␣
↪function
    kinetic_energy = 0.0

    # Initialising energy to 0 to start with - will sum over later on in the␣
↪function
    gravitational_potential_energy = 0.0

    # Double indexing to iterate over all the bodies, with i+1 included to␣
↪prevent counting any of the bodies twice
    # Using the same plummer softened distance as the acceleration
    # Using potential formula from Takayuki R. Saitoh, Junichiro Makino 2011␣
↪https://arxiv.org/pdf/1005.1752 (see equation 1 in the paper)
    for i in range(num_bodies):

        # Calculating kinetic energy
        kinetic_energy += 0.5 * masses[i] * np.square(np.linalg.
↪norm(velocities_storage_array[i]))

        for j in range(i+1, num_bodies):

            # Initial distance
            distance = np.linalg.norm(positions_storage_array[j] -␣
↪positions_storage_array[i])

            # Squaring the distance for use with formula
            distance_squared = distance**2

            # Squaring the softening parameter for use with formula
            soft_squared = soft**2

            # Computing softened plummer distance
            softened_distance = (distance_squared + soft_squared)**0.5

            # Calculating gravitational potential energy
            gravitational_potential_energy -= G * masses[i] * masses[j] /␣
↪softened_distance

    total_energy = kinetic_energy + gravitational_potential_energy

    return total_energy
```

14 Function to calculate the period

[11]:
```
# Calculating period by comparing positions over time
def calculate_period(initial_positions, positions_storage_array, timestep,␣
↪nsteps, period_tol):

    """Calculates the true, fundamental period of an n-body system (not a␣
↪harmonic) by exploiting the fact that at each fundamental period, the bodies␣
↪will be
    very close (within a small tolerance) to their starting positions. Period␣
↪calculation is multiplying the value of the first index at which
    this tolerance is met by the value of the timestep."""

    # Getting intial positions
    initial_pos = initial_positions

    # The first few steps (so the first 5%) should be ignored to stop false␣
↪positives
    steps_min = int(0.05*nsteps)

    # Getting distances from initial positions
    # Axis = 2 ensures that the array does not collapse into a 1d array which␣
↪would cause issues
    # 3d subtraction is enforced from the indexing
    distances = np.linalg.norm(positions_storage_array - initial_pos[None, :, :
↪], axis=2)

    # System only returns when the maximum value of the distance from the␣
↪initial positions across all the bodies is small
    maximum_dist_all_bodies = np.max(distances, axis=1)

    # Getting period indices
    periodic_indices = np.where(maximum_dist_all_bodies < period_tol)[0]
    periodic_indices = periodic_indices[periodic_indices >= steps_min]

    # Returning None if the period-guessing algorithm cannot calculate the␣
↪period.
    # If the algorithm fails it is because the bodies never return to their␣
↪positions within a tolerance.
    # Typically caused by large timestep values, large softening constants or␣
↪ejection
    if len(periodic_indices) == 0:

        return None

    # First return is the first periodic index
```

```
    first_return_index = periodic_indices[0]

    # Multiply timestep by first return index to get period
    period = timestep * (first_return_index+1)

    return period
```

15 Master function to run the integrator

[12]:
```
# Master function to run the integrator (uses all the core physics functions to␣
 ↪run the integrator and plot the results)
def run_integrator(parameter_file, **kwargs):

    """Runs the integrator and stores the positions, accelerations and␣
 ↪velocities as arrays for later use and returns the plots"""

    # Reading in parameters from the file
    masses, initial_positions, initial_velocities, G, soft, timestep, nsteps,␣
 ↪period_tol, filename = read_three_body_file(parameter_file)

    # Extracting initial conditions from the function call
    positions = initial_positions.copy()
    velocities = initial_velocities.copy()
    accelerations = compute_acceleration(masses, positions, G, soft)

    # Getting mode for displaying
    mode = kwargs.get('mode', None)

    ## Sanity checks - Integrator will not run if there are issues with the␣
 ↪arrays or plot display config ##

    if mode != 'close' and mode != None:
        raise ValueError('Incorrect mode entered, please only enter close or␣
 ↪leave it blank in order to display plots')

    if positions.ndim != velocities.ndim:
        raise ValueError('Dimensional inconsistency between positions and␣
 ↪velocities.')

    elif len(masses) != positions.shape[0]:
        raise ValueError('Inconsistency between number of masses and given␣
 ↪positions.')

    elif len(masses) != velocities.shape[0]:
        raise ValueError('Inconsistency between number of masses and given␣
 ↪velocities.')
```

```
  else:

      # Empty lists to store the positions, accelerations and velocities of␣
↪the bodies
      positions_storage_list = []
      velocities_storage_list = []

      # Progress bar for integrator
      for i in tqdm(range(nsteps), desc='Running integrator'):

          # Running integrator
          positions, velocities, accelerations = verlet_integration(masses,␣
↪positions, velocities, accelerations, G, soft, timestep)

          # Appending the positions after every integrator step
          positions_storage_list.append(positions)
          velocities_storage_list.append(velocities)

      # Turning the lists into arrays
      positions_storage_array = np.asarray(positions_storage_list)
      velocities_storage_array = np.asarray(velocities_storage_list)

      ## Angular momentum and total energy calculations ##
      total_system_angular_momentum_list = []
      total_energy_list = []

      # Calculating total system angular momentum and total system energy
      for position, velocity in tqdm(zip(positions_storage_array,␣
↪velocities_storage_array), desc='Calculating angular momentum and total␣
↪energy'):

          # Angular momentum calculations
          angular_momentum_magnitude = calculate_angular_momentum(masses,␣
↪position, velocity)
          total_system_angular_momentum_list.
↪append(angular_momentum_magnitude)

          # Total energy calculations
          total_energy = calculate_total_energy(masses, position, velocity,␣
↪G, soft)
          total_energy_list.append(total_energy)

      # Converting lists to arrays
      total_system_angular_momentum_array = np.
↪asarray(total_system_angular_momentum_list)
```

```
        total_energy_array = np.asarray(total_energy_list)

        # Plotting and formatting angular momentum conservation graph
        fig_angular_momentum, ax_angular_momentum = plt.
↪subplots(figsize=(12,6), dpi=300)
        ax_angular_momentum.plot(range(nsteps),␣
↪total_system_angular_momentum_array)

        ax_angular_momentum.set_title(f'Total angular momentum conservation␣
↪plot for {filename} n-body system')
        ax_angular_momentum.set_xlabel('Simulation timestep (s)')
        ax_angular_momentum.set_ylabel('Total angular momentum (kgm\u00B2/s)')
        ax_angular_momentum.grid(ls=':', color='0.9')
        fig_angular_momentum.savefig(f'Total angular momentum conservation plot␣
↪for {filename} n-body system.png')

        # Plotting and formatting total energy conservation graph
        fig_total_energy, ax_total_energy = plt.subplots(figsize=(12,6),␣
↪dpi=300)
        ax_total_energy.plot(range(nsteps), total_energy_array)

        ax_total_energy.set_title(f'Total energy conservation plot for␣
↪{filename} n-body system')
        ax_total_energy.set_xlabel('Number of simulation timesteps')
        ax_total_energy.set_ylabel('Total energy (J)')
        ax_total_energy.grid(ls=':', color='0.9')
        fig_total_energy.savefig(f'Total energy conservation plot for␣
↪{filename} n-body system')

        # Calculating period
        period = calculate_period(initial_positions, positions_storage_array,␣
↪timestep, nsteps, period_tol)

        ## 3D Plotting section ##

        # Making 3D figure
        fig = plt.figure(dpi=300, figsize=(4,4))
        ax = fig.add_subplot(projection='3d')

        # Setting the axis limit based on the position values
        fig_range = np.max(positions_storage_array)*1.2
        ax.set_xlim(-fig_range, fig_range)
        ax.set_ylim(-fig_range, fig_range)
        ax.set_zlim(-fig_range, fig_range)

        # Formatting labels
```

```
        ax.set_title(f'{filename} trajectories, nsteps={nsteps},␣
↪timestep={timestep}, soft={soft}, period_tol={period_tol}')
        ax.set_xlabel('X / m')
        ax.set_ylabel('Y / m')
        ax.set_zlabel('Z / m')

        # Plotting filter for fast dragging

        for mass in range(len(masses)):

            # Extracting positions
            x_coord = positions_storage_array[:, mass, 0]
            y_coord = positions_storage_array[:, mass, 1]
            z_coord = positions_storage_array[:, mass, 2]

            # Plotting trajectories (no label here)
            line = ax.plot(x_coord, y_coord, z_coord, linewidth=0.8)

            # Plotting body final positions
            ax.scatter(*positions_storage_array[-1, mass], s=50, marker='o',␣
↪color=line[0].get_color(), label=f'Particle {mass + 1}')

        # Adding the legend and using the tight layout for better formatting
        ax.legend()
        plt.tight_layout()

        # Saving final configuration
        fig.savefig(f'{filename} trajectories, nsteps={nsteps},␣
↪timestep={timestep}, soft={soft}.png')

        # This will be used on the validation plots to conserve memory
        if mode == 'close':
            plt.close('all')

        # If no mode is defined, force the plots to display
        elif mode == None:
            plt.show('all')

        # Printing median energy and median angular momentum.
        # Median is statistically better for averaging over large datasets␣
↪because it is anomaly-resistant
        print(f'Median system total energy is {np.median(total_energy_array)}␣
↪J')
        print(f'Median system angular momentum is {np.
↪median(total_system_angular_momentum_array)} kgm\u00B2/s')
        print(f'Period is {period}(s)')
```

```
    return fig, ax, period
```

16 Function to run parallelised validation of Kepler's 3rd law using the two body problem

[13]:
```
def run_parallelised_kepler_two_body_validation(parameter_file):

    """Validates Kepler's 3rd law using this process:

    1. Runs the integrator n_cores times for 2-body systems, each with a different value of the semi major axis from the semi major axis array
    2. Extracts the period from the parallelised integration runs.
    3. For the systems, uses the real kepler's 3rd law and the semi major axis to calculate the theoretical values for the periods.
    4. Calculates the percentage errors with the numerical periods vs the theoretical ones.
    5. Plots a graph of T**2 against R**3, using the real kepler constant as a gradient and the numerically computed squares of the periods."""

    # Reading in file
    mass_1, mass_2, dist_min, dist_max, num_tests, G, soft, period_tol, system_type, mode = read_two_body_file(parameter_file)

    # Test distances
    test_distances = np.linspace(dist_min, dist_max, num_tests)

    ## Sanity check on system_type arguement ##

    if system_type != 'star-planet' and system_type != 'elliptical-binary':
        raise ValueError('Incorrect or no system type entered. Please only call this function with star-planet or elliptical-binary entered')

    # Setting up masses
    masses = np.array([mass_1, mass_2], dtype='float64')
    total_mass = np.sum(masses)

    # Empty arrays to capture outputs
    numerical_periods_list = []
    theoretical_periods_list = []

    # Empty arrays to store per-run parameters
    initial_positions_list = []
    initial_velocities_list = []
    timestep_sizes_list = []
```

```
    nsteps_list = []
    filenames_list = []

    ## For every test distance, calculate a theoretical and numerical period␣
↪for star-planet and set up parallelised run parameters ##

    if system_type == 'star-planet':
        for distance, i in zip(test_distances, range(num_tests)):

            # Calculating theoretical period using kepler's 3rd law
            theoretical_period = np.sqrt(4 * np.pi**2 * distance**3 / (G *␣
↪total_mass))

            # Set up circular orbit's initial conditions for the integrator
            # So numerical period can be captured

            # Distance from the COM
            COM_r1 = mass_2 * distance / total_mass
            COM_r2 = mass_1 * distance / total_mass

            # Circular orbit velocities
            # Using keplerian velocity
            v_relative = np.sqrt(G * total_mass / distance)
            v1 = mass_2 * v_relative / total_mass
            v2 = mass_1 * v_relative / total_mass

            # ICS for integrator
            initial_positions = np.array([[-COM_r1, 0, 0], [COM_r2, 0, 0]],␣
↪dtype='float64')
            initial_velocities = np.array([[0, -v1, 0], [0, v2, 0]],␣
↪dtype='float64')

            # Around 10000 timesteps
            timestep = theoretical_period / 10000

            # Number of steps is proportional to the theoretical period
            nsteps = int(2 * theoretical_period / timestep)

            # Formatting so graphs for system visualisation and conservation of␣
↪angular momentum and energy are not lost
            filename = f'star-planet run {i}'

            # Capture calculated parameters
            initial_positions_list.append(initial_positions)
            initial_velocities_list.append(initial_velocities)
            timestep_sizes_list.append(timestep)
            nsteps_list.append(nsteps)
```

```
            theoretical_periods_list.append(theoretical_period)
            filenames_list.append(filename)

    elif system_type == 'elliptical-binary':

        # Sanity check for equal masses - It is an elliptical binary system
        if mass_1 != mass_2:
            raise ValueError('An elliptical binary system configuration has␣
↪equal masses, so please run with equal masses.')

        else:
            for distance, i in zip(test_distances, range(num_tests)):

                # Calculating theoretical period using kepler's 3rd law
                theoretical_period = np.sqrt(4 * np.pi**2 * distance**3 / (G *␣
↪total_mass))

                # Set up elliptical system initial conditions using vis-viva:␣
↪https://en.wikipedia.org/wiki/Vis-viva_equation
                # Eccentricity parameter
                e = 0.5

                # Starting at closest points
                r_closest = distance * (1 - e)

                # Distance from the COM for both bodies
                COM_r1 = mass_1 * r_closest / total_mass
                COM_r2 = mass_2 * r_closest / total_mass

                # Velocities
                v_relative = np.sqrt(G * total_mass * (2/r_closest - 1/
↪distance))
                v_1 = v_relative * COM_r1 / r_closest
                v_2 = v_relative * COM_r2 / r_closest

                # ICS for integrator
                initial_velocities = np.array([[0, v_1, 0], [0, -v_1, 0]],␣
↪dtype='float64')
                initial_positions = np.array([[COM_r1, 0, 0], [-COM_r2, 0, 0]],␣
↪dtype='float64')

                # Around 10000 timesteps
                timestep = theoretical_period / 10000

                # Number of steps is proportional to the theoretical period
                nsteps = int(2 * theoretical_period / timestep)
```

```
                # Formatting so graphs for conservation of angular momentum and␣
↪energy are not lost
                filename = f'elliptical-binary run {i}'

                # Capture calculated parameters
                initial_positions_list.append(initial_positions)
                initial_velocities_list.append(initial_velocities)
                timestep_sizes_list.append(timestep)
                nsteps_list.append(nsteps)
                theoretical_periods_list.append(theoretical_period)
                filenames_list.append(filename)

    ## Setting up parallelised runs ##

    # Converting to numpy arrays
    initial_positions_array = np.asarray(initial_positions_list)
    initial_velocities_array = np.asarray(initial_velocities_list)
    timestep_sizes_array = np.asarray(timestep_sizes_list)
    nsteps_array = np.asarray(nsteps_list)

    # Setting up per-orbit run parameters
    single_orbit_run_args = [(masses, initial_positions_list[i],␣
↪initial_velocities_list[i], G, soft, timestep_sizes_list[i], nsteps_list[i],␣
↪period_tol, filenames_list[i]) for i in range(num_tests)]

    # Running the simulations in parallel across all detected CPU cores
    with Pool(processes=os.cpu_count()) as pool:

        trajectory_figs_axs_periods_list = list(tqdm(pool.
↪imap(run_integrator_worker, single_orbit_run_args), total=num_tests,␣
↪desc='Running parallelised 2-body problem simulations'))

    # Unpacking the results from the parallelised runs

    trajectory_figs, trajectory_axs, numerical_periods_list =␣
↪zip(*trajectory_figs_axs_periods_list)

    # Convert the unpacked numerical period to an array, and account for any␣
↪NaNs that crop up if period guessing fails
    numerical_periods_array = np.array([np.nan if period is None else period␣
↪for period in numerical_periods_list], dtype='float64')

    # Convert the theoretical period setup earlier to an array
    theoretical_periods_array = np.asarray(theoretical_periods_list)
```

```
    ## Plotting validation results ##

    # Sanity checks for numerical results
    if numerical_periods_array.size == 0 or np.all(np.
↪isnan(numerical_periods_array)):
        raise ValueError('Numerical period not computed, validation therefore␣
↪not possible.')

    elif theoretical_periods_array.shape != numerical_periods_array.shape:
        raise ValueError('Numerical period array and theoretical period array␣
↪dimensions are not the same.')

    # Warnings for runs where no period was found
    failed_period_calculation_runs = np.count_nonzero(np.
↪isnan(numerical_periods_array))
    if failed_period_calculation_runs > 0:
        warnings.warn(f'Warning, the period calculation function failed to␣
↪return a period for {failed_period_calculation_runs} of {num_tests}␣
↪parallelised runs.\nThis is because the bodies never returned to their␣
↪initial positions within the period_tol parameter in the parameter file.')

    # Validation plots
    fig, ax = plt.subplots(1, 3, figsize=(14, 4), dpi=300)

    # Getting theoretical and numerically computed periods, and the distances
    T_squared_theoretical = theoretical_periods_array**2
    T_squared_numerical = numerical_periods_array**2
    R_cubed = test_distances**3

    # Computing percentage errors
    percentage_errors = 100 * np.abs(numerical_periods_array -␣
↪theoretical_periods_array) / theoretical_periods_array

    # First set of axes - plotting kepler's 3rd law
    ax[0].plot(R_cubed, T_squared_theoretical, color='black', linestyle='--',␣
↪label='Theoretical Kepler\'s 3rd Law fit')
    ax[0].scatter(R_cubed, T_squared_numerical, color='red', marker='x',␣
↪label='Numerical Kepler\'s 3rd law fit')
    ax[0].set_xlabel('R\u00B3')
    ax[0].set_ylabel('T\u00B2')
    ax[0].set_title('T\u00B2 vs R\u00B3 Plot')
    ax[0].grid(ls=':', color='0.9')

    # Second set of axes - Agreement plot
```

```
    ax[1].plot([0, np.max(theoretical_periods_array)], [0, np.
↪max(theoretical_periods_array)], linestyle='--', color='black',␣
↪label='Period agreement')
    ax[1].scatter(numerical_periods_array, theoretical_periods_array,␣
↪color='red', marker='x', label='Numerical periods')
    ax[1].set_xlabel('Numerical period (s)')
    ax[1].set_ylabel('Theoretical period (s)')
    ax[1].set_title('Numerical vs theoretical period agreement plot')
    ax[1].grid(ls=':', color='0.9')

    # Third set of axes - Percentage error from residuals
    ax[2].scatter(test_distances, percentage_errors, color='red', marker='x',␣
↪label='Numerical period error')
    ax[2].axhline(y=0, color='black', linestyle='--')
    ax[2].set_xlabel('Test distance between bodies (m)')
    ax[2].set_ylabel('Percentage error (dimensionless)')
    ax[2].set_title('Integrator accuracy plot')
    ax[2].grid(ls=':', color='0.9')

    # Formatting
    ax[0].legend()
    ax[1].legend()
    ax[2].legend()
    fig.suptitle(f'2-body validation plot for {system_type}')
    plt.tight_layout()

    # Calculating and printing the theoretical and numerical kepler constant␣
↪agreement
    theoretical_kepler_constant = 4 * np.pi**2 / (G * total_mass)
    numerical_kepler_constant = np.nanmean(T_squared_numerical / R_cubed)
    overall_error_percentage = 100*np.abs((numerical_kepler_constant -␣
↪theoretical_kepler_constant) / theoretical_kepler_constant)

    # Printing error for kepler constant
    print("----------")
    print(f'Final percentage error for numerical Kepler constant against␣
↪theoretical Kepler constant = {overall_error_percentage}%')
    print("----------")

    # Saving validation plots
    plt.savefig(f'Two-body problem validation {system_type}.png')
    plt.show()

    ## Displaying the final trajectory plot ##
    print("----------")
    print(f'Displaying the final trajectory plot for system type {system_type}')
```

```
    print("----------")

    # Recovering and displaying the orbit configuration of the final␣
↪parallelised validation run as an image
    plt.show(trajectory_figs[-1])

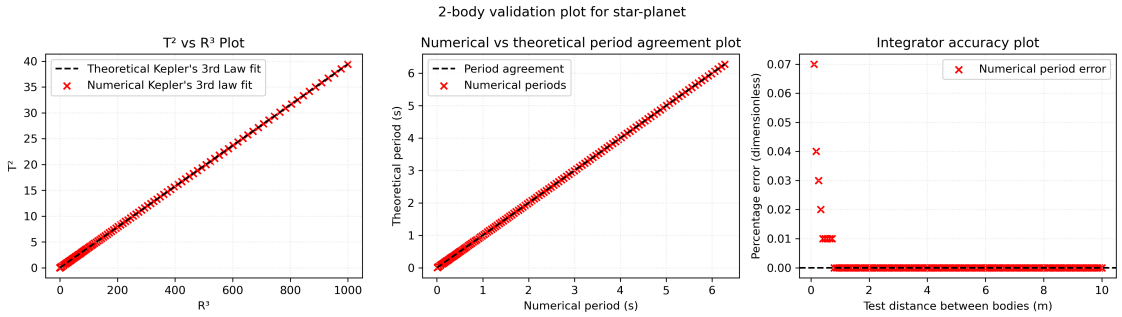
    return fig, ax
```

17 Two body validation runs

[14]:
```
# Two body validation for circular orbit systems
run_parallelised_kepler_two_body_validation('star-planet_ICS.txt')
```

```
Running parallelised 2-body problem simulations: 100%|      | 128/128
[00:13<00:00,  9.24it/s]

----------
Final percentage error for numerical Kepler constant against theoretical Kepler
constant = 0.0032806015625229637%
----------
```



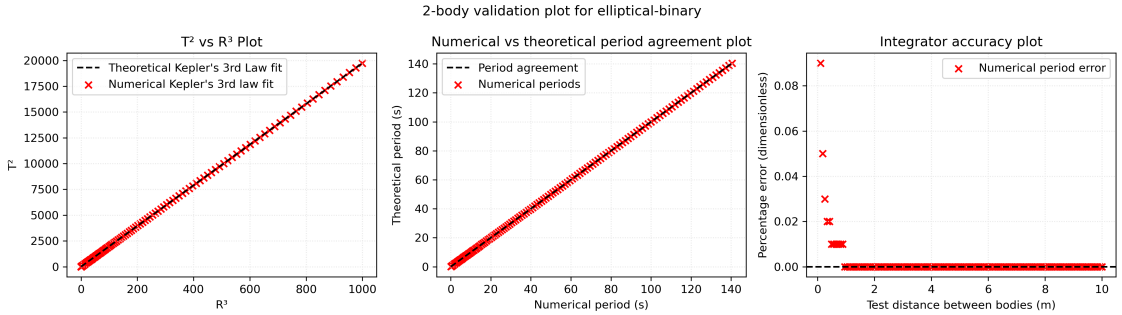

```
----------
Displaying the final trajectory plot for system type star-planet
----------
```

[14]:
```
(<Figure size 4200x1200 with 3 Axes>,
 array([<Axes: title={'center': 'T² vs R³ Plot'}, xlabel='R³', ylabel='T²'>,
        <Axes: title={'center': 'Numerical vs theoretical period agreement
plot'}, xlabel='Numerical period (s)', ylabel='Theoretical period (s)'>,
        <Axes: title={'center': 'Integrator accuracy plot'}, xlabel='Test
distance between bodies (m)', ylabel='Percentage error (dimensionless)'>],
       dtype=object))
```

[15]:
```
# Two body validation for elliptical binary systems
run_parallelised_kepler_two_body_validation('elliptical-binary_ICS.txt')
```

```
Running parallelised 2-body problem simulations: 100%|      | 128/128
[00:14<00:00,  8.54it/s]

----------
Final percentage error for numerical Kepler constant against theoretical Kepler
constant = 0.00421774218747043%
----------
```



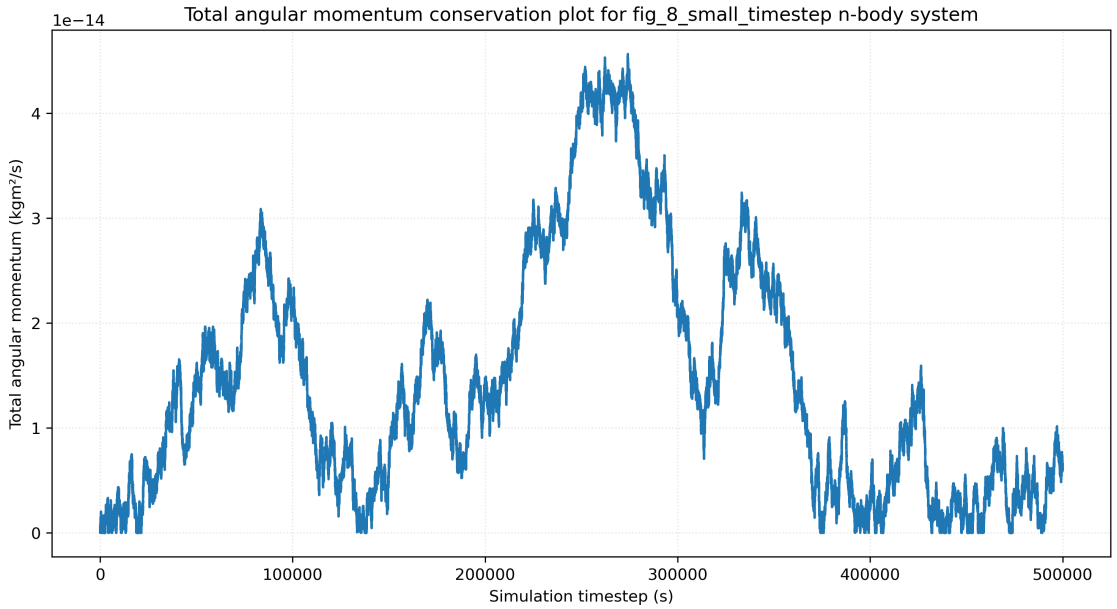

```
----------
Displaying the final trajectory plot for system type elliptical-binary
----------
```

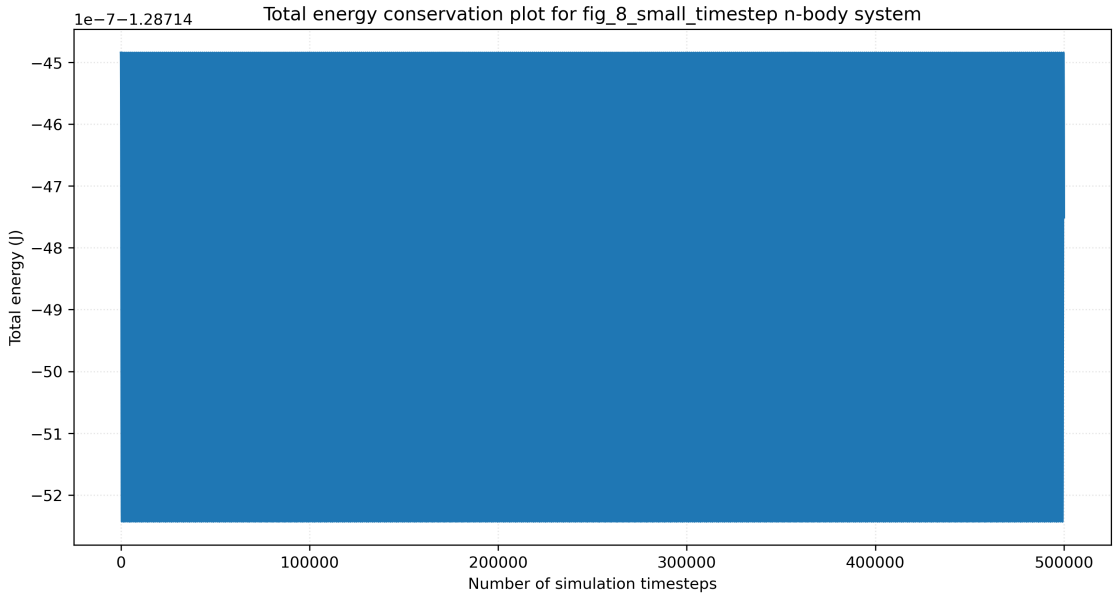
[15]:
```
(<Figure size 4200x1200 with 3 Axes>,
 array([<Axes: title={'center': 'T² vs R³ Plot'}, xlabel='R³', ylabel='T²'>,
        <Axes: title={'center': 'Numerical vs theoretical period agreement
plot'}, xlabel='Numerical period (s)', ylabel='Theoretical period (s)'>,
        <Axes: title={'center': 'Integrator accuracy plot'}, xlabel='Test
distance between bodies (m)', ylabel='Percentage error (dimensionless)'>],
       dtype=object))
```

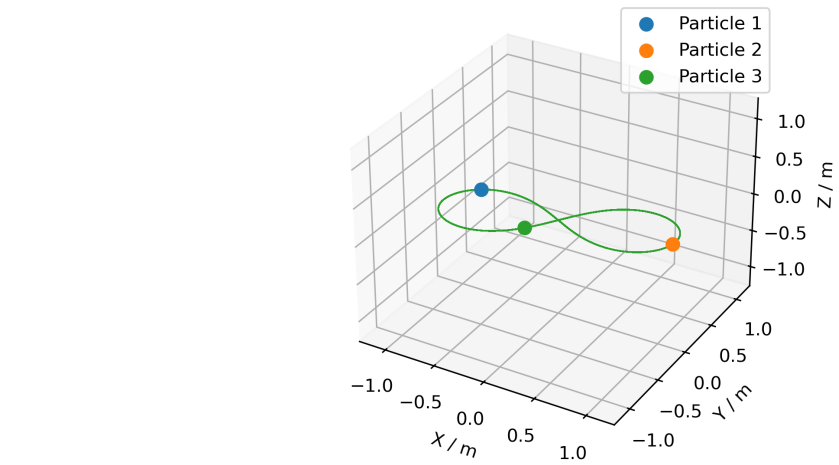
18 Convergence dependence on timestep value

[16]:
```
# Small timestep
run_integrator('fig_8_convergence_small_timestep_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 29722.62it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23290.05it/s]
```


Total angular momentum conservation plot for fig_8_small_timestep n-body system
1e−14
Total angular momentum (kgm²/s)
0
1
2
3
4
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8_small_timestep n-body system
1e−7−1.28714
Total energy (J)
−45
−46
−47
−48
−49
−50
−51
−52
0
100000
200000
300000
400000
500000
Number of simulation timesteps

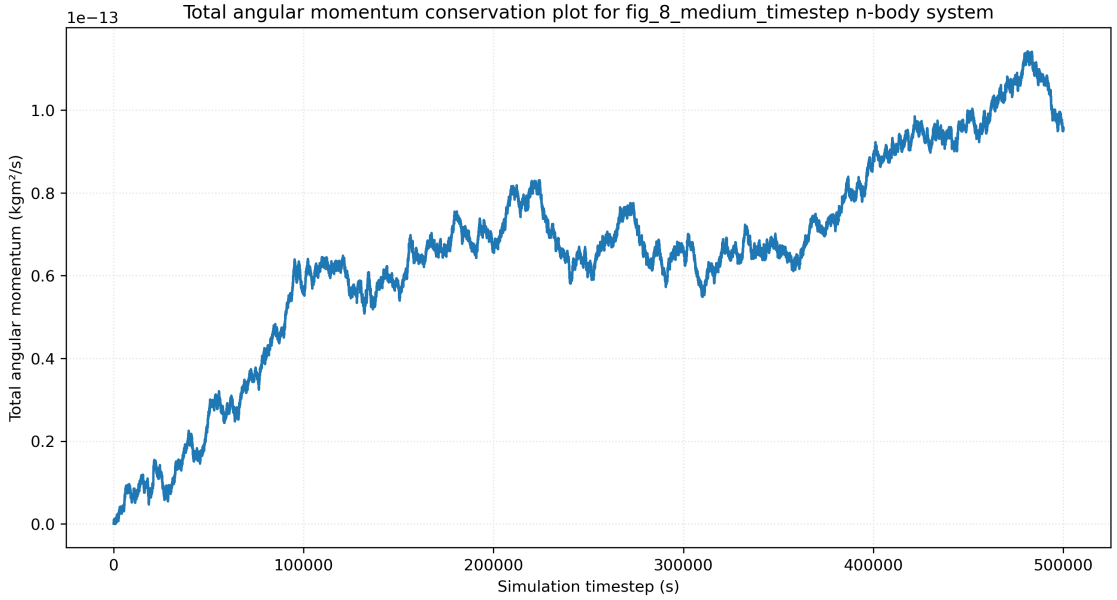


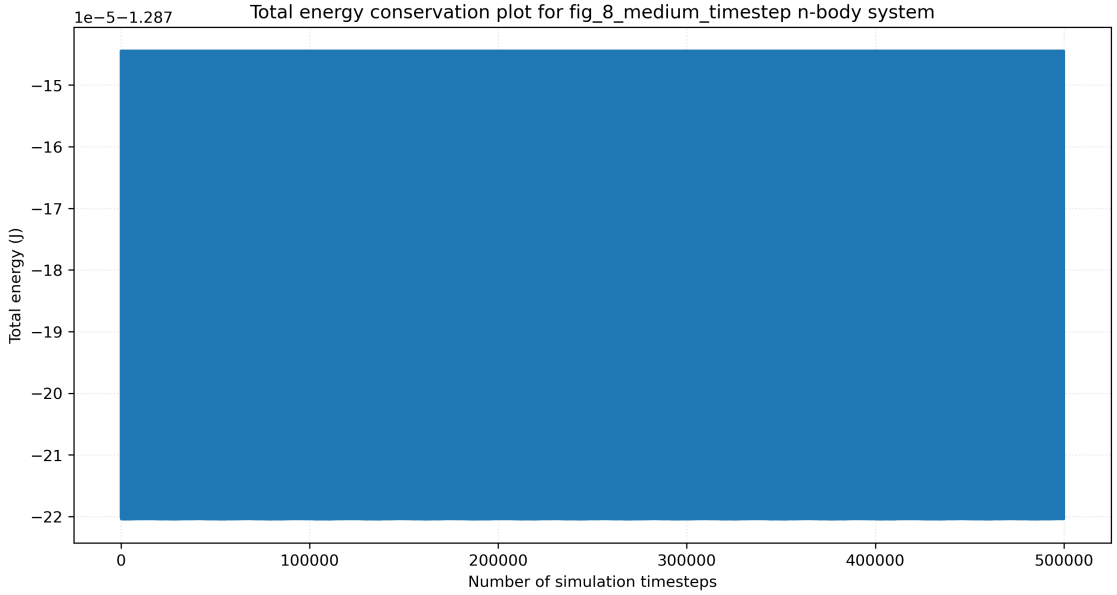

```
Median system total energy is -1.2871447707954795 J
Median system angular momentum is 1.2406742300186124e-14 kgm²/s
Period is 44.281(s)
```

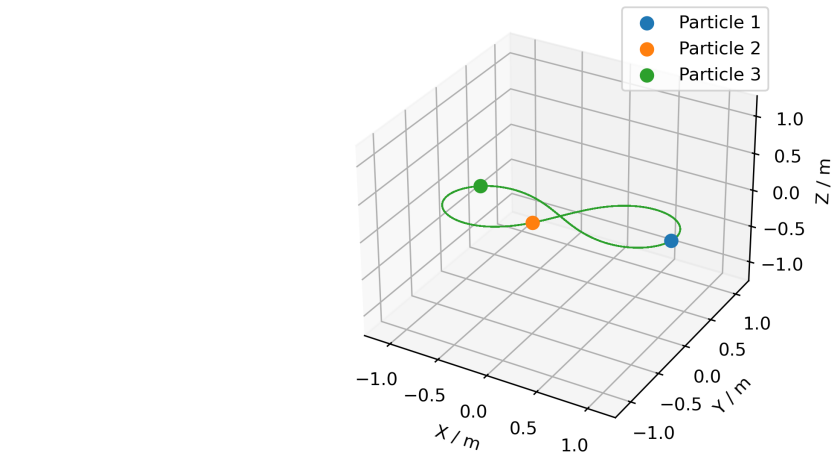
[16]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_small_timestep trajectories, nsteps=500000,
timestep=0.001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(44.281))
```

[17]:
```
# Medium timestep
run_integrator('fig_8_convergence_medium_timestep_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30111.23it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23482.69it/s]
```


Total angular momentum conservation plot for fig_8_medium_timestep n-body system
1e−13
Total angular momentum (kgm²/s)
1.0
0.8
0.6
0.4
0.2
0.0
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8_medium_timestep n-body system
1e−5−1.287
Total energy (J)
−15
−16
−17
−18
−19
−20
−21
−22
0
100000
200000
300000
400000
500000
Number of simulation timesteps

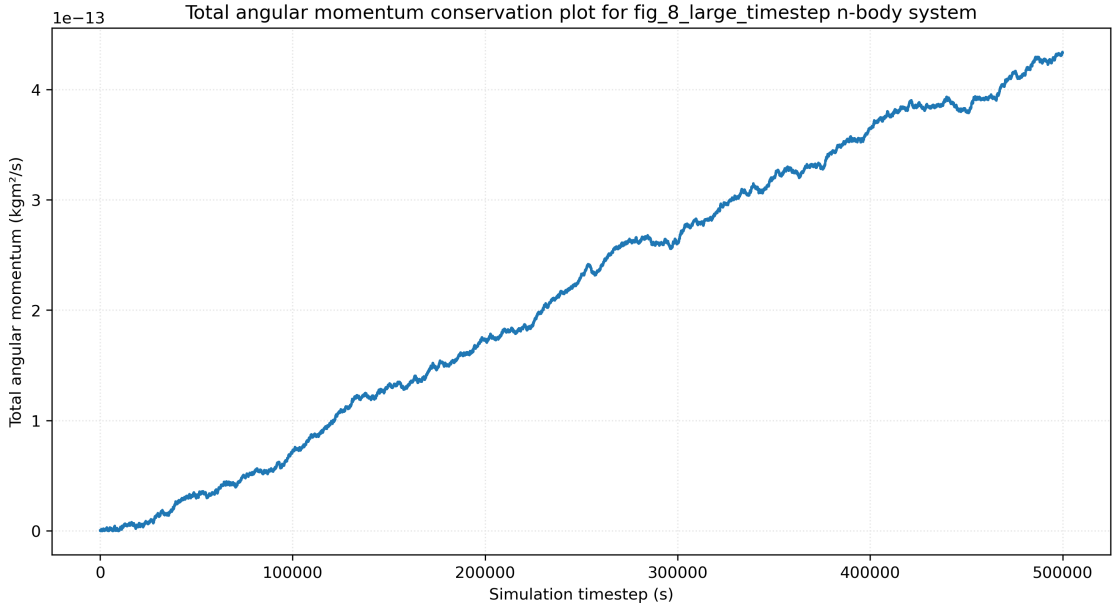



```
Median system total energy is -1.2871731310408374 J
Median system angular momentum is 6.639133687258436e-14 kgm²/s
Period is 594.66(s)
```

[17]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_medium_timestep trajectories, nsteps=500000,
timestep=0.01, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m',
zlabel='Z / m'>,
 np.float64(594.66))
```

[18]:
```
# Large timestep
run_integrator('fig_8_convergence_large_timestep_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30166.56it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23473.94it/s]
```


Total angular momentum conservation plot for fig_8_large_timestep n-body system
1e−13
Total angular momentum (kgm²/s)
0
1
2
3
4
0
100000
200000
300000
400000
500000
Simulation timestep (s)

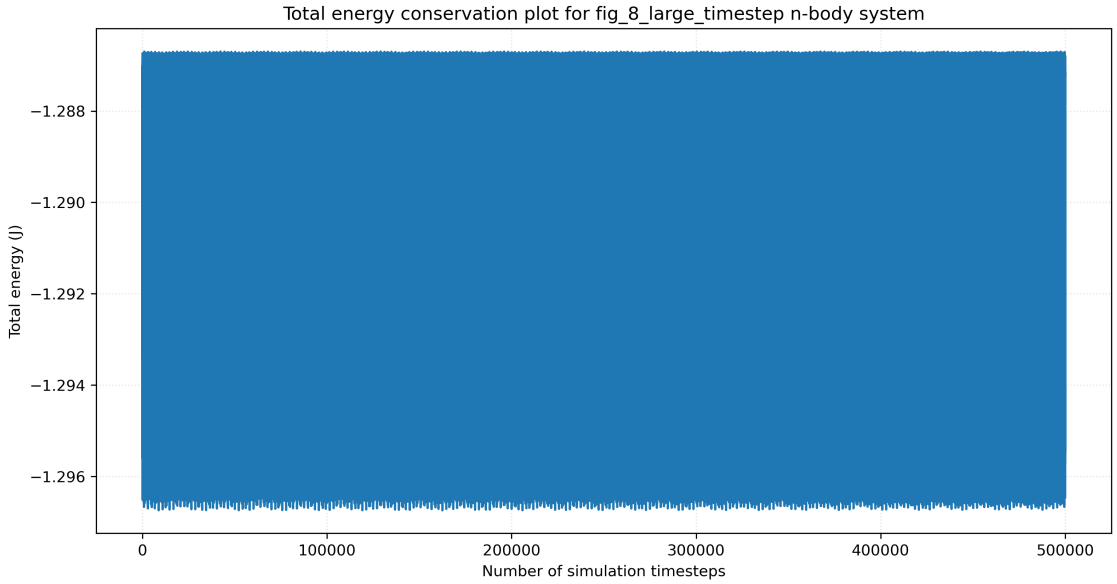
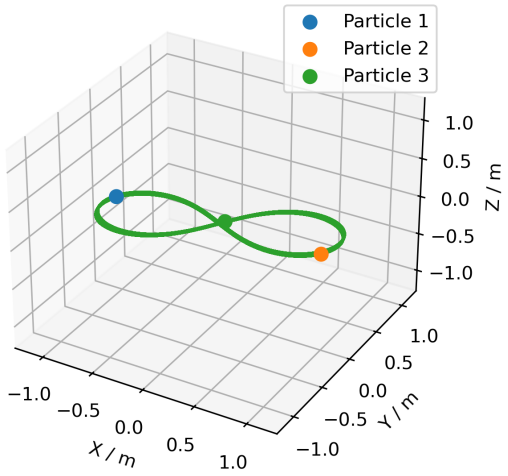

Total energy conservation plot for fig_8_large_timestep n-body system
Total energy (J)
−1.288
−1.290
−1.292
−1.294
−1.296
0
100000
200000
300000
400000
500000
Number of simulation timesteps

fig_8_large_timestep trajectories, nsteps=500000, timestep=0.1, soft=0.0001, period_tol=0.0005



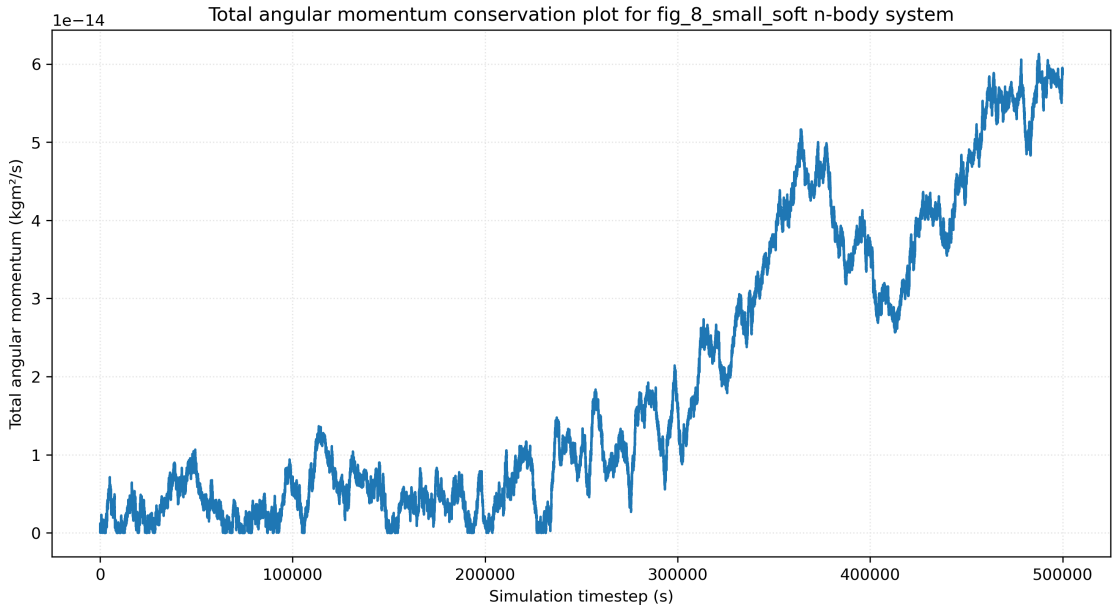

```
Median system total energy is -1.2898439788245089 J
Median system angular momentum is 2.3166191187584673e-13 kgm²/s
Period is None(s)
```

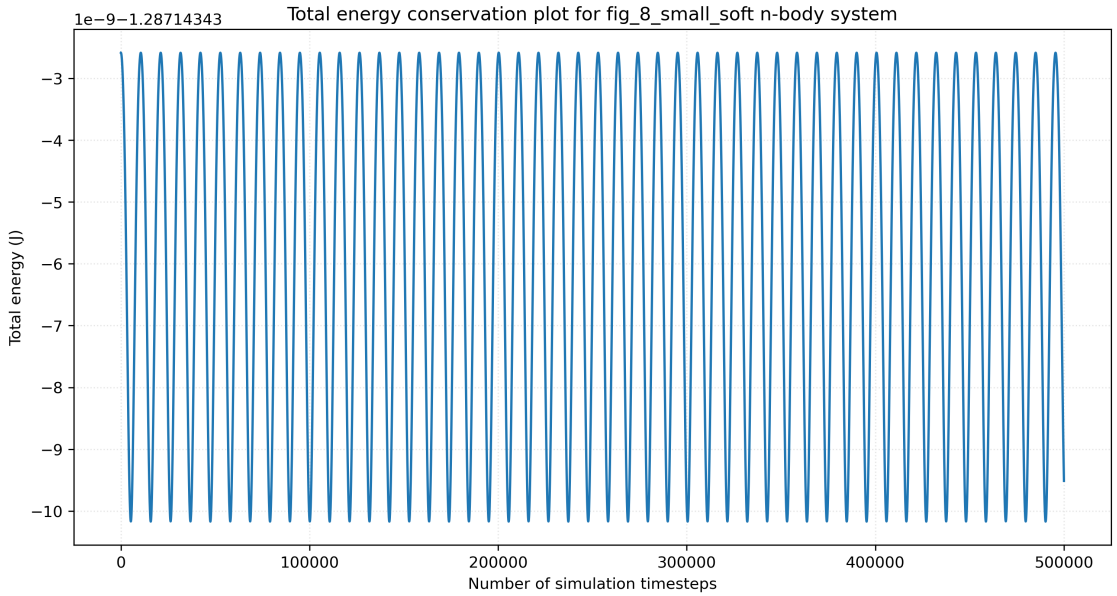
[18]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_large_timestep trajectories, nsteps=500000,
timestep=0.1, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m',
zlabel='Z / m'>,
 None)
```

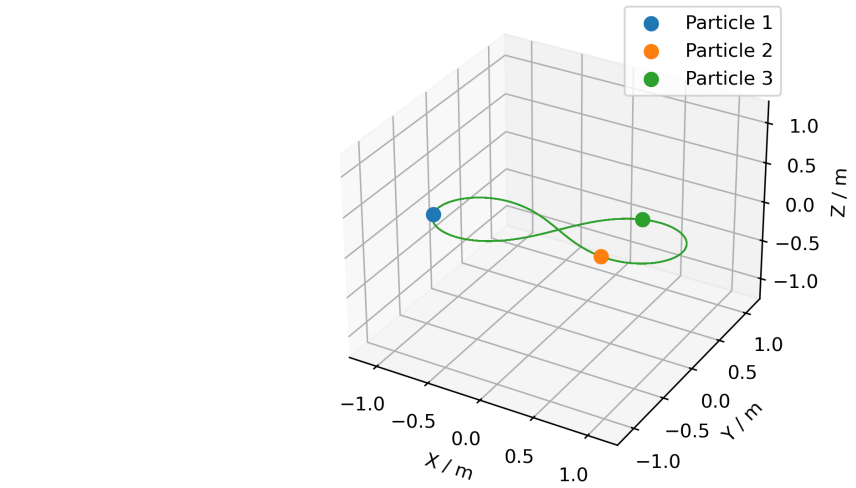
19 Convergence dependence on softening value

[19]:
```
# Small softening constant
run_integrator('fig_8_convergence_small_soft_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 29907.83it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23477.67it/s]
```


Total angular momentum conservation plot for fig_8_small_soft n-body system
1e−14
Total angular momentum (kgm²/s)
0
1
2
3
4
5
6
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8_small_soft n-body system
1e−9−1.28714343
Total energy (J)
−3
−4
−5
−6
−7
−8
−9
−10
0
100000
200000
300000
400000
500000
Number of simulation timesteps

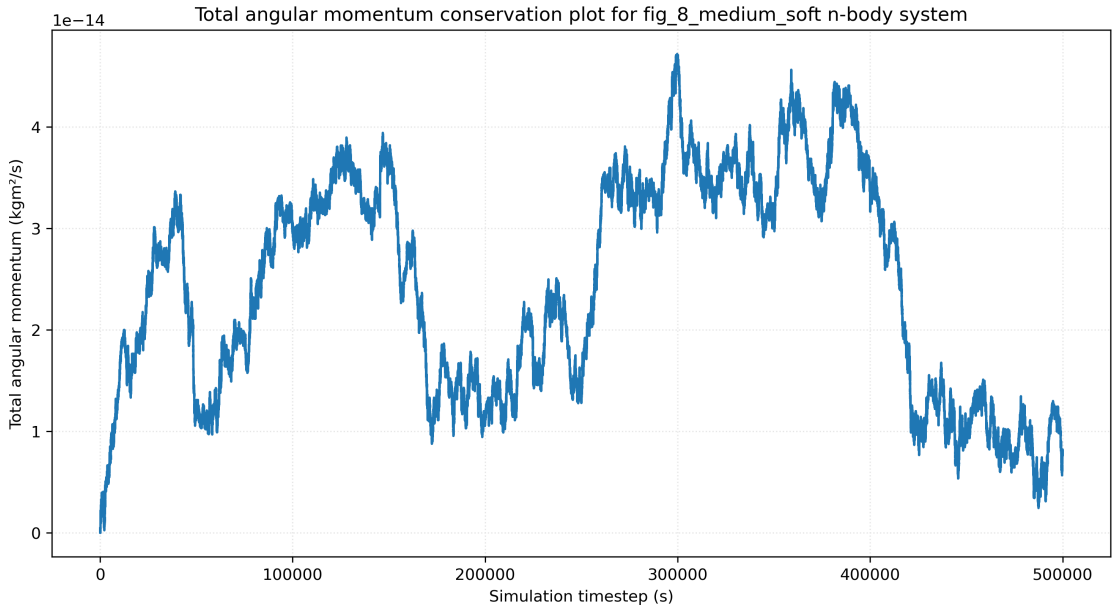


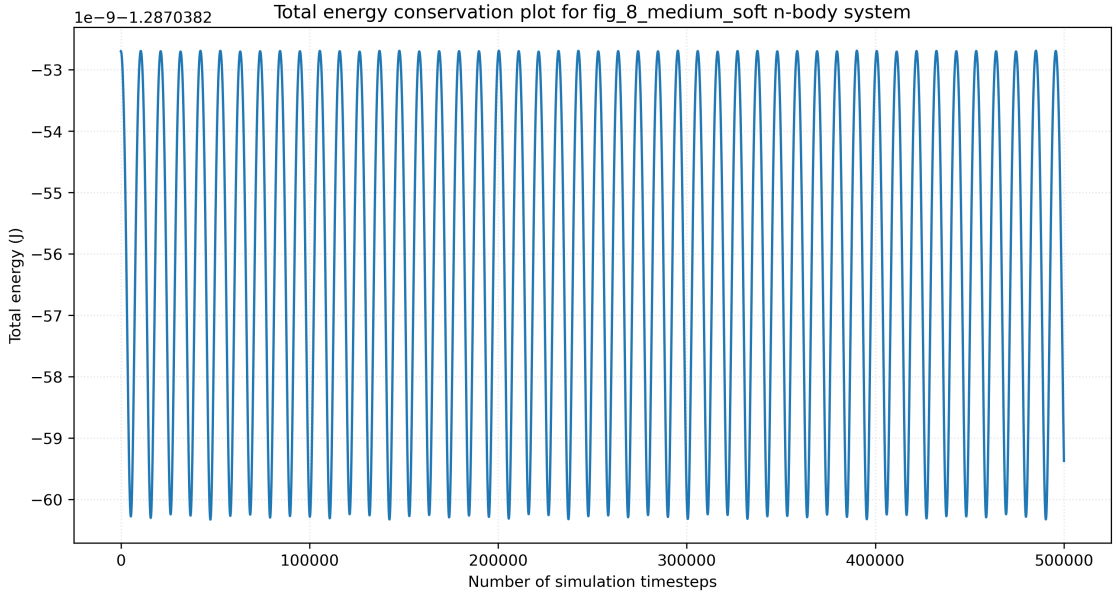

```
Median system total energy is -1.2871434354428128 J
Median system angular momentum is 1.0047518372857667e-14 kgm²/s
Period is 6.3256000000000006(s)
```

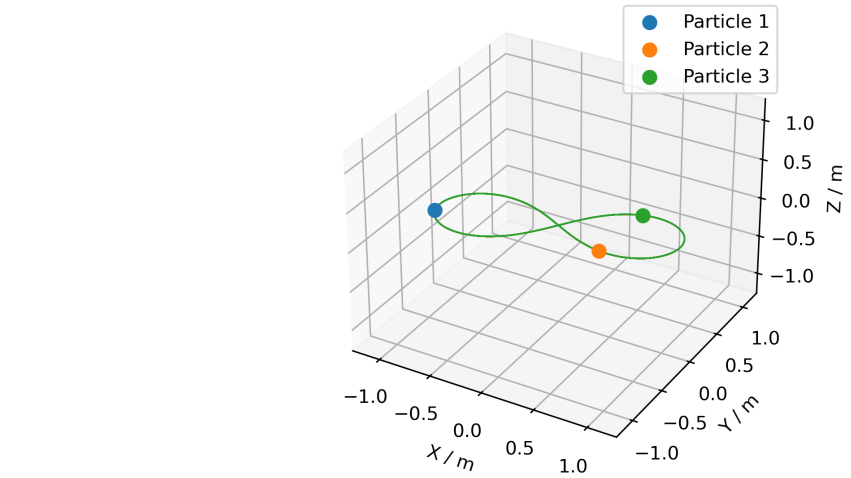
[19]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_small_soft trajectories, nsteps=500000,
timestep=0.0001, soft=0.001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(6.3256000000000006))
```

[20]:
```
# Medium softening constant
run_integrator('fig_8_convergence_medium_soft_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30011.57it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23264.75it/s]
```


Total angular momentum conservation plot for fig_8_medium_soft n-body system
1e−14
Total angular momentum (kgm²/s)
0
1
2
3
4
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8_medium_soft n-body system
1e−9−1.2870382
Total energy (J)
−53
−54
−55
−56
−57
−58
−59
−60
0
100000
200000
300000
400000
500000
Number of simulation timesteps

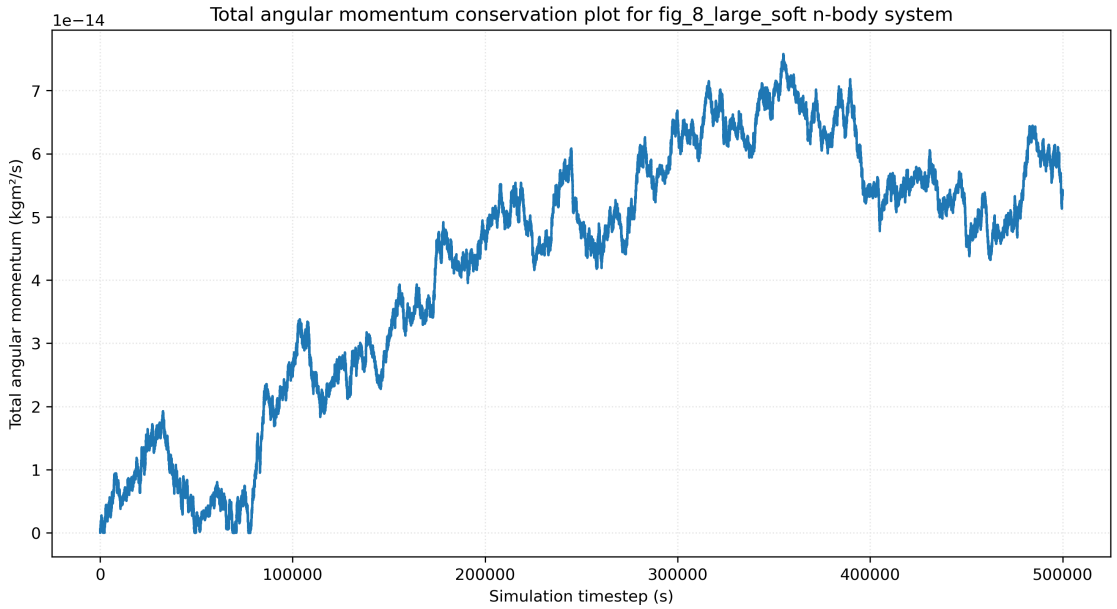


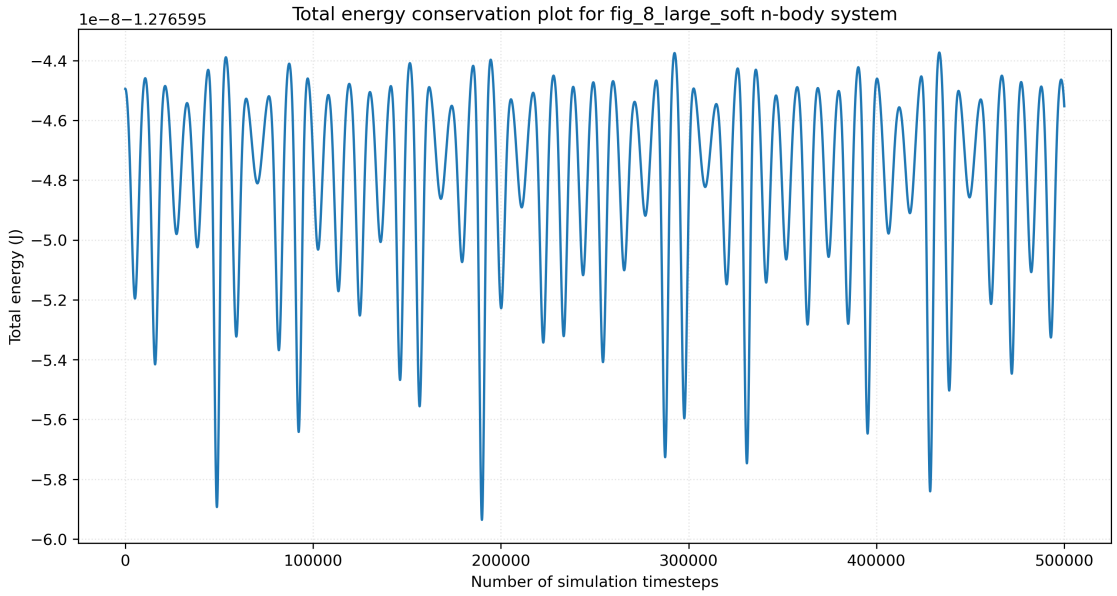

```
Median system total energy is -1.2870382555539852 J
Median system angular momentum is 2.5757174171303632e-14 kgm²/s
Period is 6.3269(s)
```

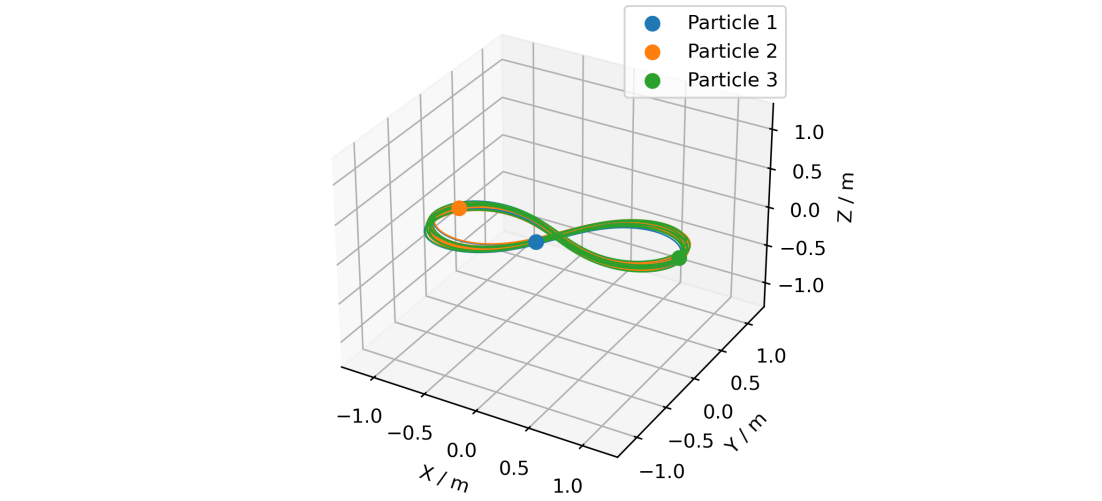
[20]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_medium_soft trajectories, nsteps=500000,
timestep=0.0001, soft=0.01, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m',
zlabel='Z / m'>,
 np.float64(6.3269))
```

[21]:
```
# Large softening constant
run_integrator('fig_8_convergence_large_soft_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30100.19it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23352.01it/s]
```


Total angular momentum conservation plot for fig_8_large_soft n-body system
1e−14
Total angular momentum (kgm²/s)
0
1
2
3
4
5
6
7
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8_large_soft n-body system
1e−8−1.276595
Total energy (J)
−4.4
−4.6
−4.8
−5.0
−5.2
−5.4
−5.6
−5.8
−6.0
0
100000
200000
300000
400000
500000
Number of simulation timesteps




```
Median system total energy is -1.2765950472964815 J
Median system angular momentum is 4.873879078104437e-14 kgm²/s
Period is None(s)
```

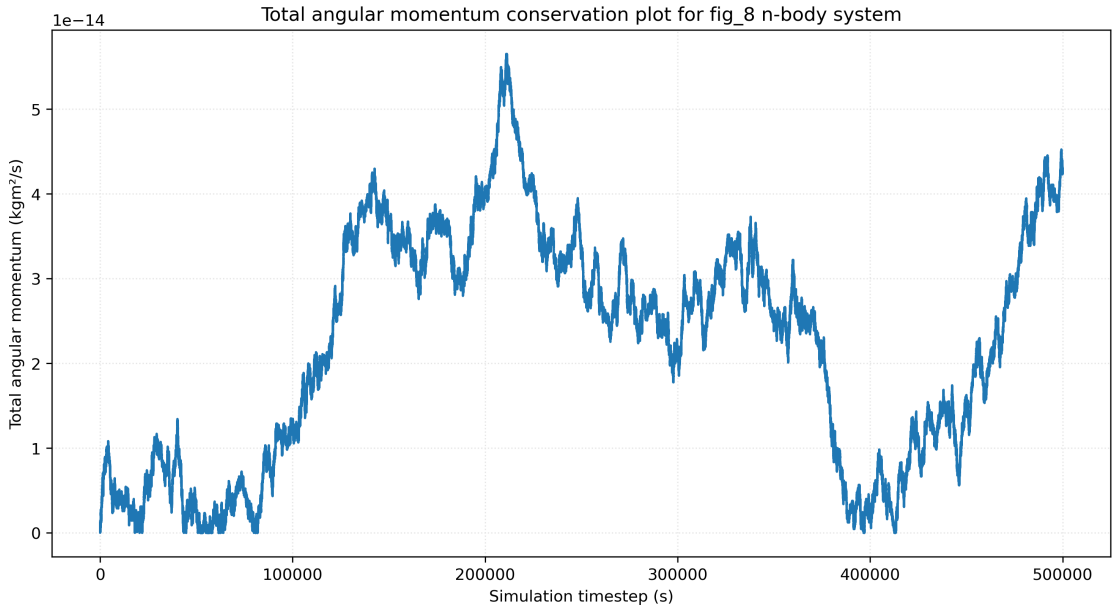
[21]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8_large_soft trajectories, nsteps=500000,
timestep=0.0001, soft=0.1, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m',
zlabel='Z / m'>,
 None)
```

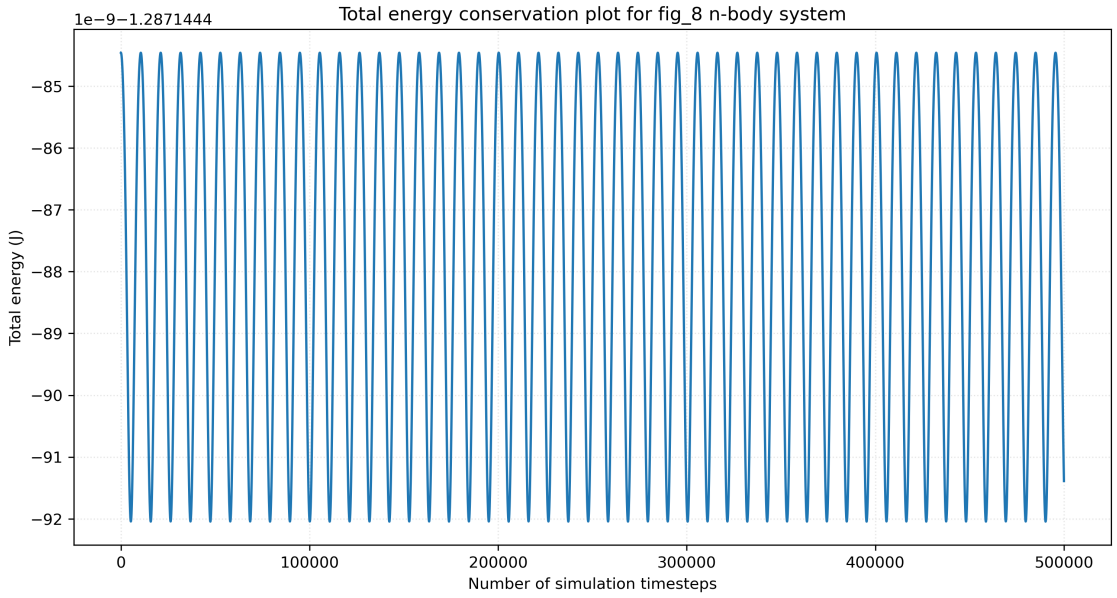
20 3-body problem plots

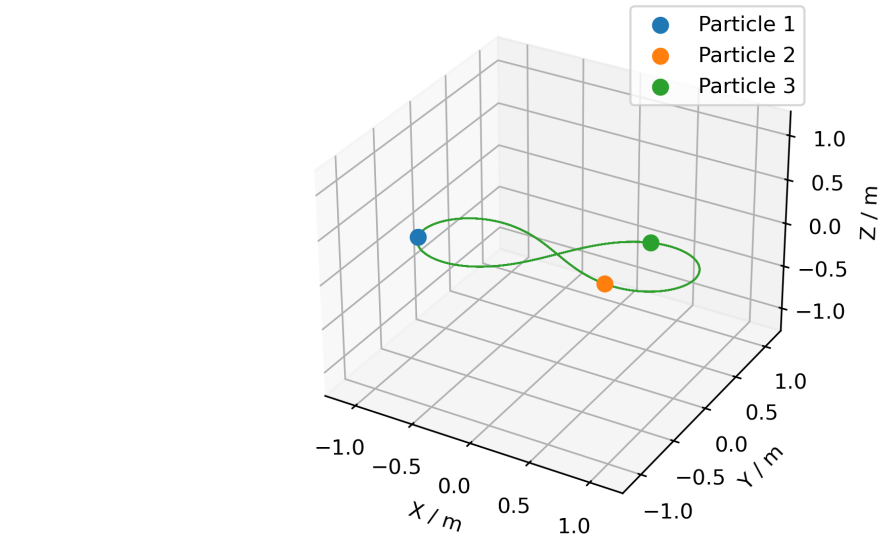
20.1 2D Planar Solutions

[22]:
```
# Figure of 8
run_integrator('fig_8_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30190.26it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23393.90it/s]
```


Total angular momentum conservation plot for fig_8 n-body system
1e−14
Total angular momentum (kgm²/s)
0
1
2
3
4
5
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for fig_8 n-body system
1e−9−1.2871444
Total energy (J)
−85
−86
−87
−88
−89
−90
−91
−92
0
100000
200000
300000
400000
500000
Number of simulation timesteps

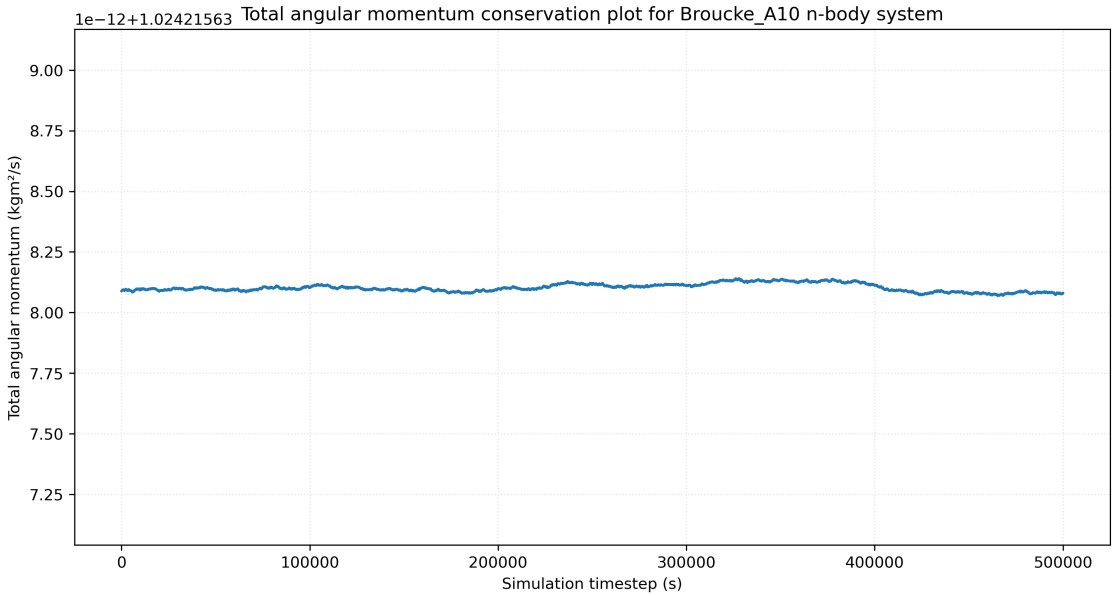


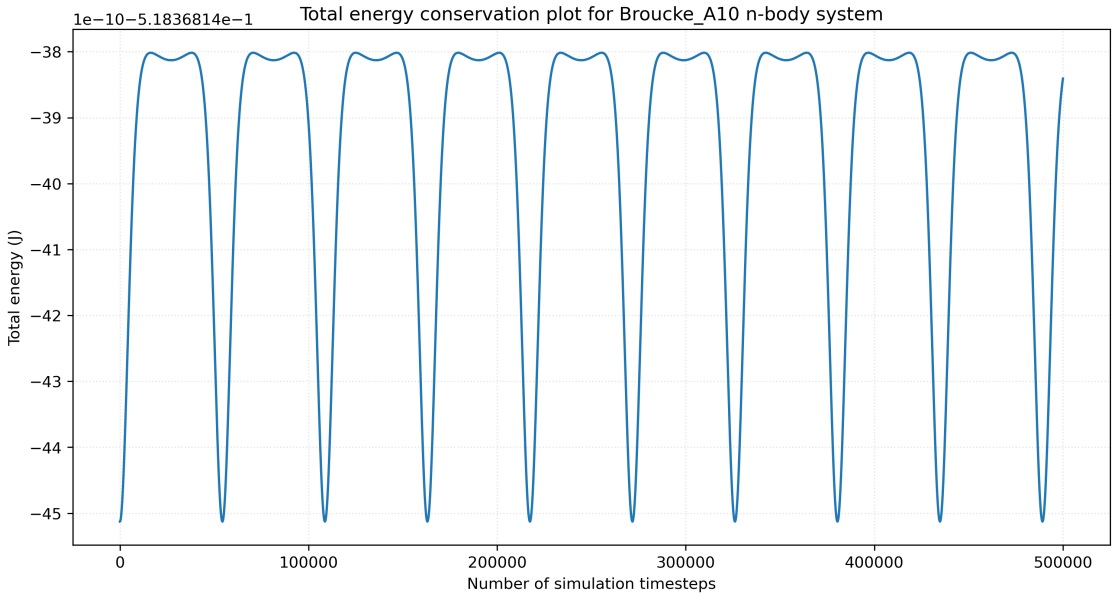

```
Median system total energy is -1.2871444873172 J
Median system angular momentum is 2.5091040356528538e-14 kgm²/s
Period is 6.3256000000000006(s)
```

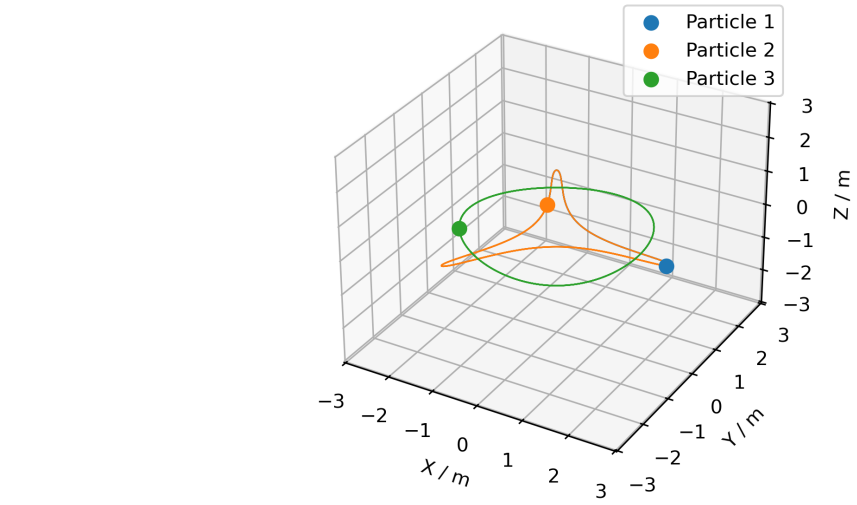
[22]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'fig_8 trajectories, nsteps=500000, timestep=0.0001,
soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m', zlabel='Z /
m'>,
 np.float64(6.3256000000000006))
```

[23]:
```
# Brouke A10
run_integrator('Broucke_A10_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30126.53it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23278.76it/s]
```


Total angular momentum conservation plot for Broucke_A10 n-body system
1e−12+1.02421563
9.00
8.75
8.50
8.25
8.00
7.75
7.50
7.25
Total angular momentum (kgm²/s)
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for Broucke_A10 n-body system
1e−10−5.1836814e−1
−38
−39
−40
−41
−42
−43
−44
−45
Total energy (J)
0
100000
200000
300000
400000
500000
Number of simulation timesteps

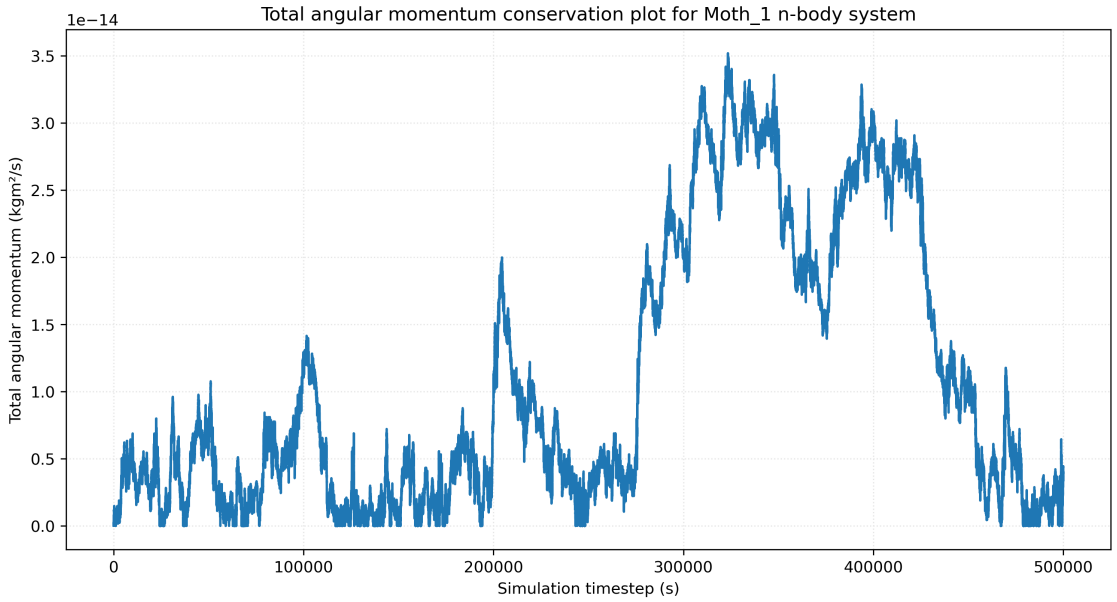


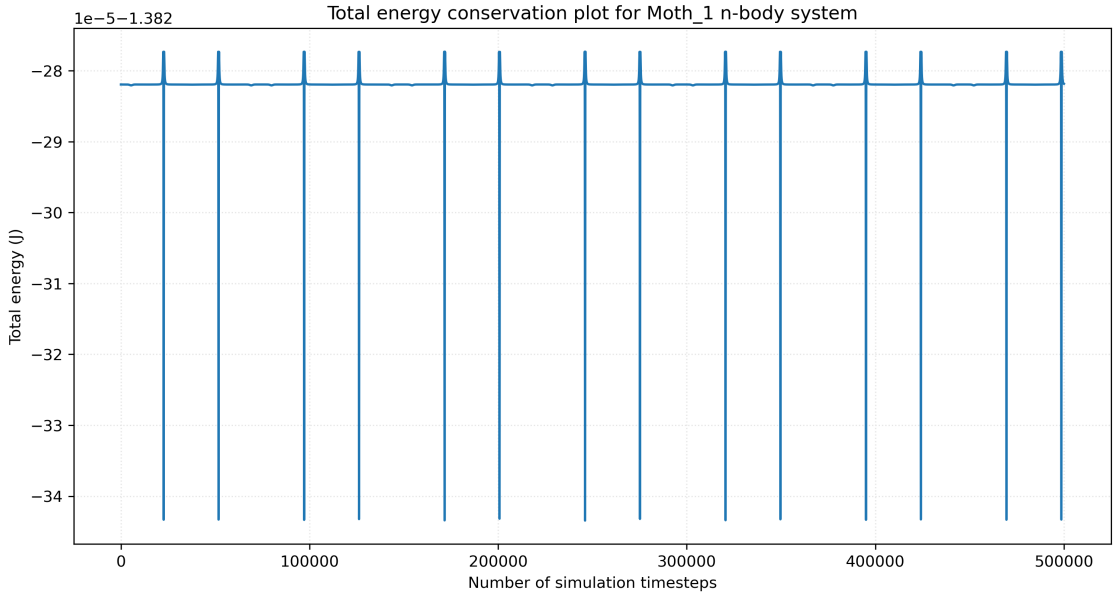

```
Median system total energy is -0.5183681438129003 J
Median system angular momentum is 1.0242156299580993 kgm²/s
Period is 32.609(s)
```

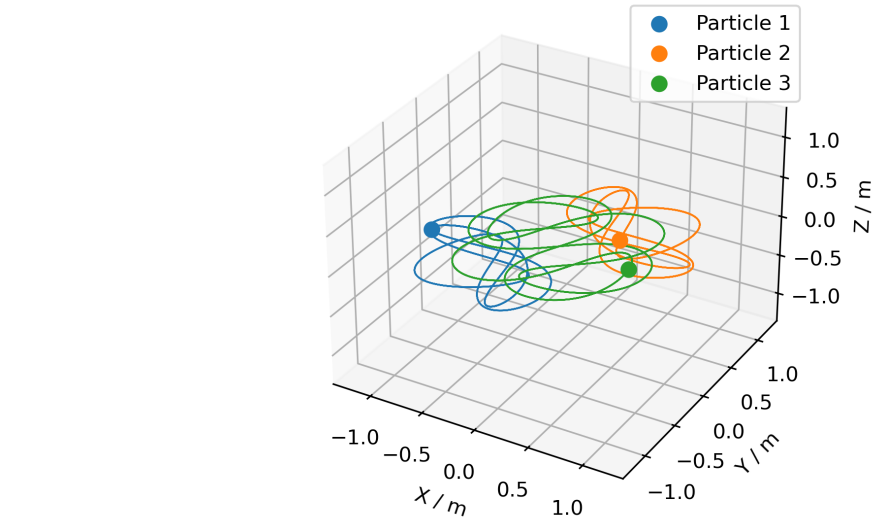
[23]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Broucke_A10 trajectories, nsteps=500000,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(32.609))
```

[24]:
```
# Moth 1
run_integrator('Moth_1_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 31216.58it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23475.13it/s]
```


Total angular momentum conservation plot for Moth_1 n-body system
1e−14
Total angular momentum (kgm²/s)
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for Moth_1 n-body system
1e−5−1.382
Total energy (J)
−28
−29
−30
−31
−32
−33
−34
0
100000
200000
300000
400000
500000
Number of simulation timesteps

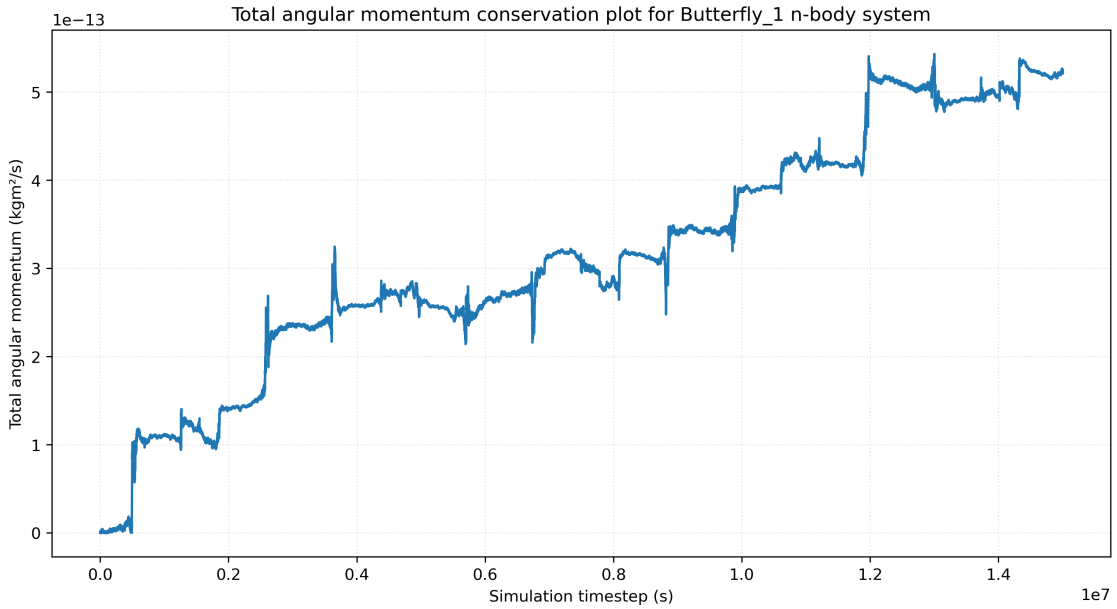


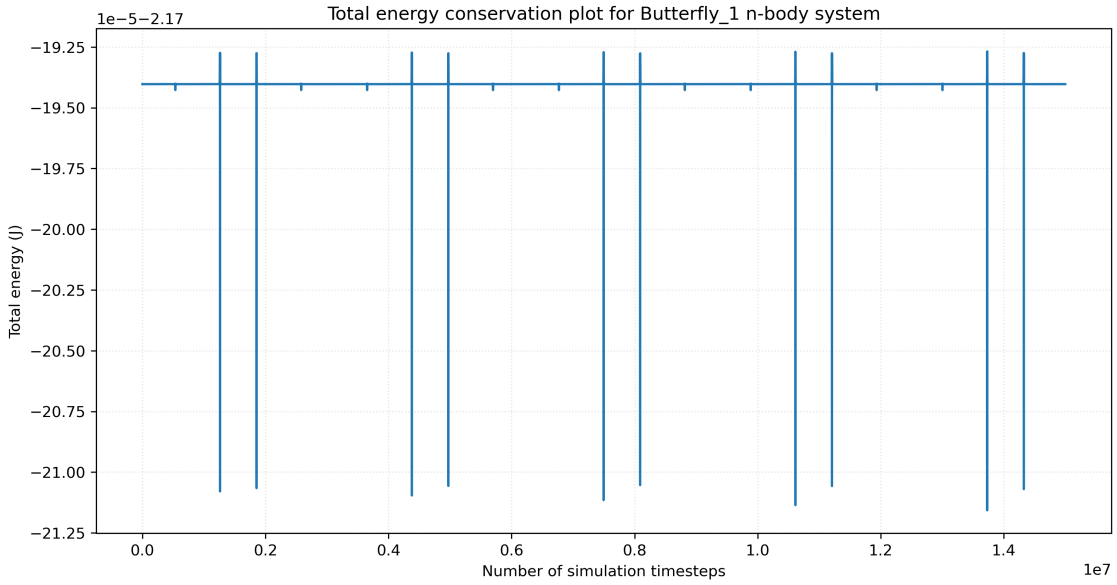

```
Median system total energy is -1.3822819476809438 J
Median system angular momentum is 5.88418203051333e-15 kgm²/s
Period is 14.8939(s)
```

[24]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Moth_1 trajectories, nsteps=500000, timestep=0.0001,
soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y / m', zlabel='Z /
m'>,
 np.float64(14.8939))
```

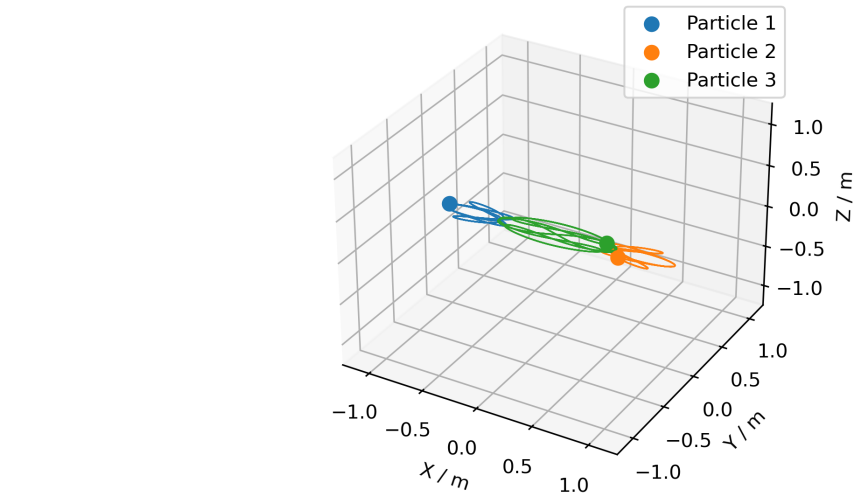
[25]:
```
# Butterfly 1
run_integrator('Butterfly_1_ICS.txt')
```

```
Running integrator: 100%|      | 15000000/15000000 [08:17<00:00,
30132.00it/s]
Calculating angular momentum and total energy: 15000000it [10:39, 23471.68it/s]
C:\Users\bksh2\AppData\Local\Temp\ipykernel_15492\1917898842.py:129:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  plt.tight_layout()
C:\Users\bksh2\AppData\Local\Temp\ipykernel_15492\1917898842.py:132:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  fig.savefig(f'{filename} trajectories, nsteps={nsteps}, timestep={timestep},
soft={soft}.png')
```






```
C:\Users\bksh2\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  fig.canvas.print_figure(bytes_io, **kw)
```

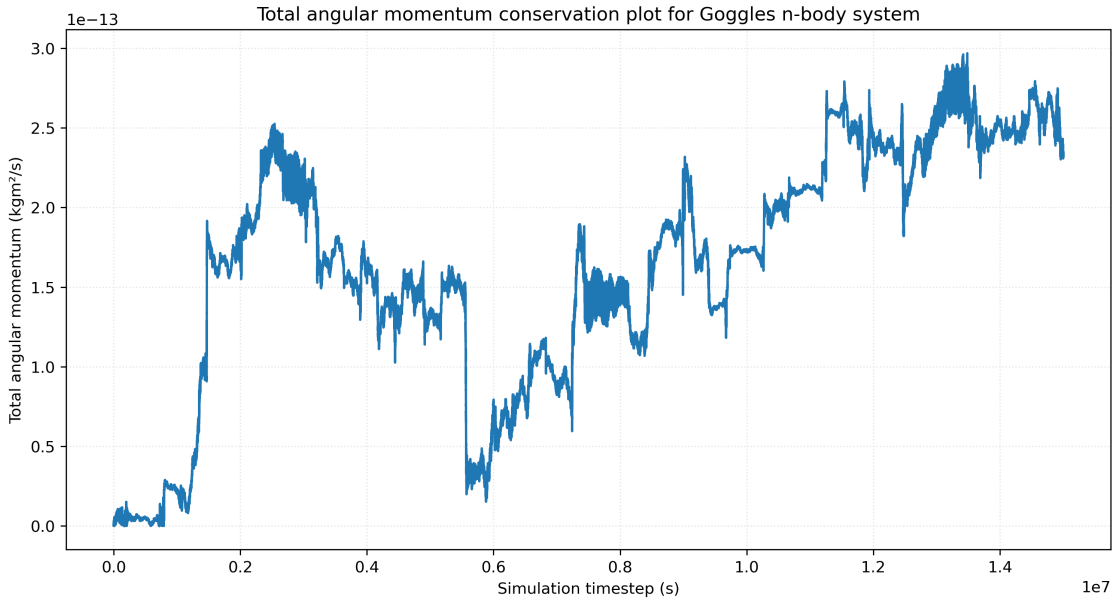


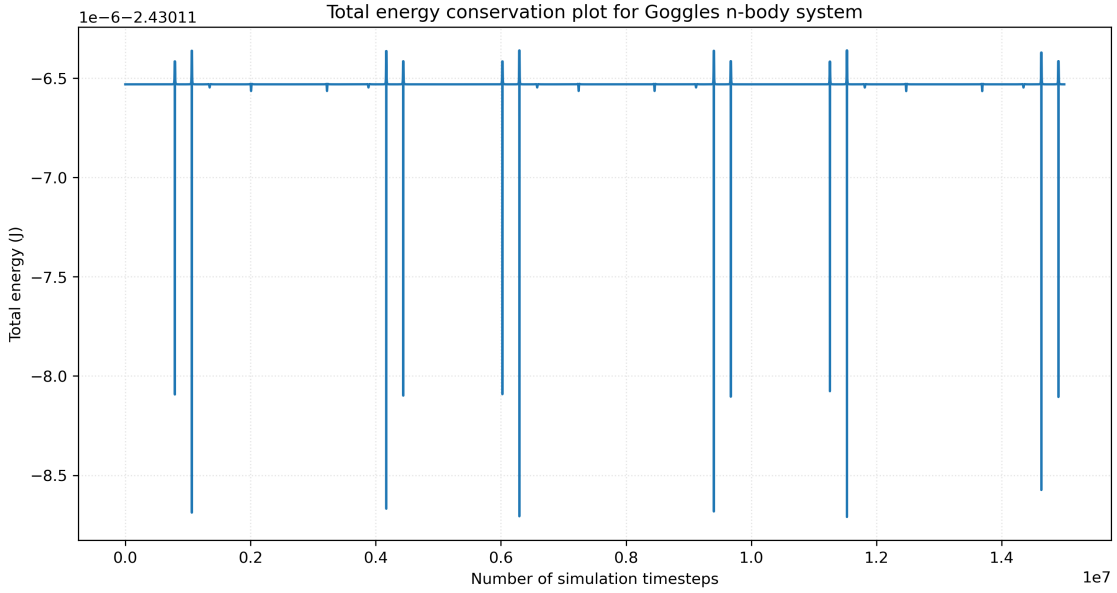

```
Median system total energy is -2.1701940278899174 J
Median system angular momentum is 3.0261904093720204e-13 kgm²/s
Period is 6.234115999999999(s)
```

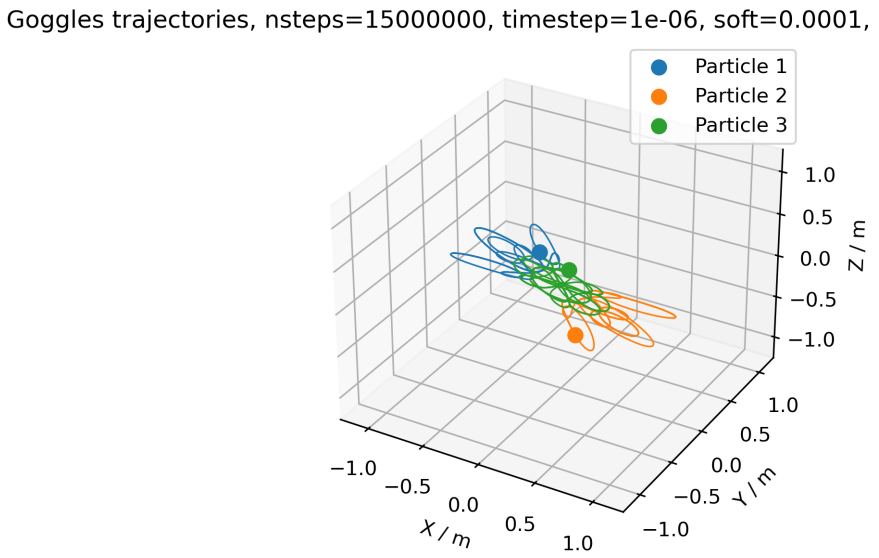
[25]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Butterfly_1 trajectories, nsteps=15000000,
timestep=1e-06, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(6.234115999999999))
```

[26]:
```
# Goggles
run_integrator('Goggles_ICS.txt')
```

```
Running integrator: 100%|      | 15000000/15000000 [08:17<00:00,
30151.92it/s]
Calculating angular momentum and total energy: 15000000it [10:38, 23486.36it/s]
C:\Users\bksh2\AppData\Local\Temp\ipykernel_15492\1917898842.py:129:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  plt.tight_layout()
C:\Users\bksh2\AppData\Local\Temp\ipykernel_15492\1917898842.py:132:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  fig.savefig(f'{filename} trajectories, nsteps={nsteps}, timestep={timestep},
soft={soft}.png')
```


Total angular momentum conservation plot for Goggles n-body system
1e−13
Total angular momentum (kgm²/s)
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1e7
Simulation timestep (s)


Total energy conservation plot for Goggles n-body system
1e−6−2.43011
Total energy (J)
−6.5
−7.0
−7.5
−8.0
−8.5
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1e7
Number of simulation timesteps




```
Median system total energy is -2.4301165303917887 J
Median system angular momentum is 1.7048341904857267e-13 kgm²/s
Period is 10.463348(s)
```

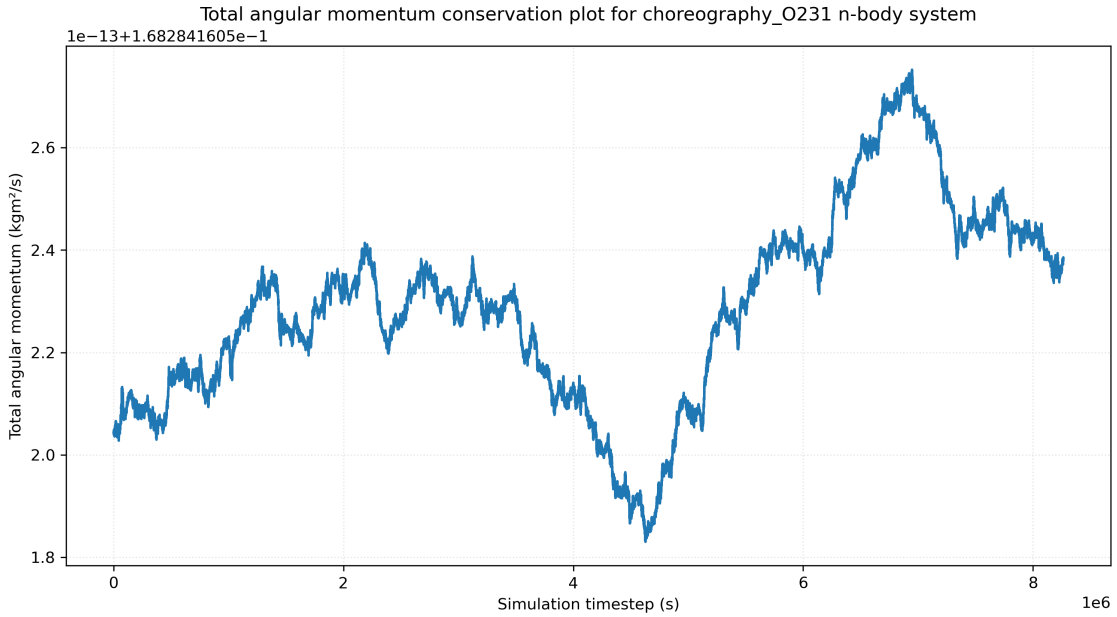
[26]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Goggles trajectories, nsteps=15000000,
timestep=1e-06, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(10.463348))
```

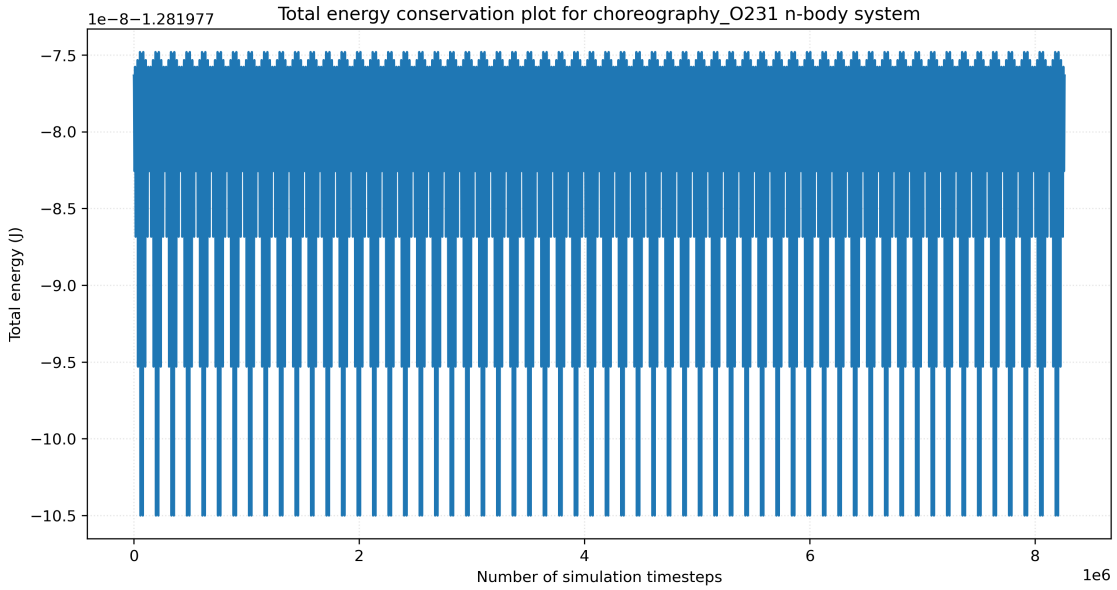
20.2 3D Solutions

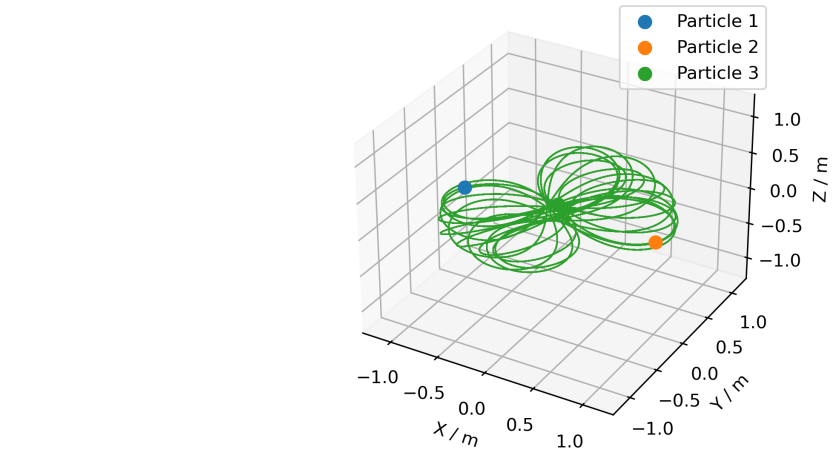
20.3 Source: X. Li and S. Liao, "Discovery of 10,059 new three-dimensional periodic orbits of general three-body problem", arXiv:2508.08568 (2025), orbit O_231(1.0).

[27]:
```
# O231(1.0) Li and Liao 2025 3D orbit
run_integrator('choreography_O231_ICS.txt')
```

```
Running integrator: 100%|      | 8263840/8263840 [04:34<00:00, 30109.24it/s]
Calculating angular momentum and total energy: 8263840it [05:52, 23437.16it/s]
C:\Users\bksh2\AppData\Local\Temp\ipykernel_15492\1917898842.py:129:
UserWarning: Creating legend with loc="best" can be slow with large amounts of
data.
  plt.tight_layout()
```


Total angular momentum conservation plot for choreography_O231 n-body system
1e−13+1.682841605e−1
Total angular momentum (kgm²/s)
2.6
2.4
2.2
2.0
1.8
0
2
4
6
8
Simulation timestep (s)
1e6


Total energy conservation plot for choreography_O231 n-body system
1e−8−1.281977
Total energy (J)
−7.5
−8.0
−8.5
−9.0
−9.5
−10.0
−10.5
0
2
4
6
8
Number of simulation timesteps
1e6

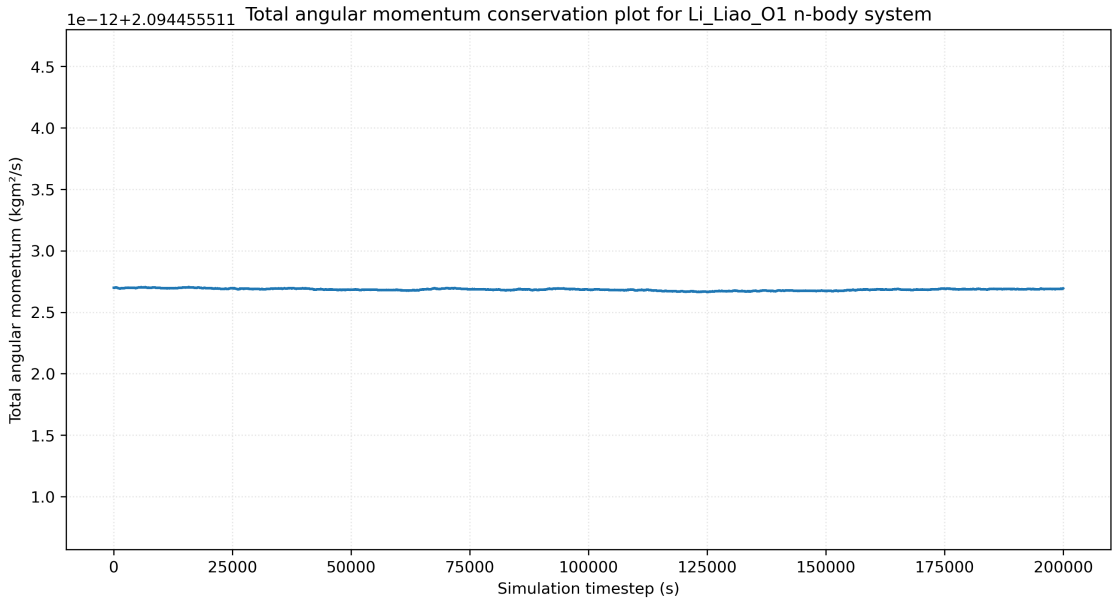


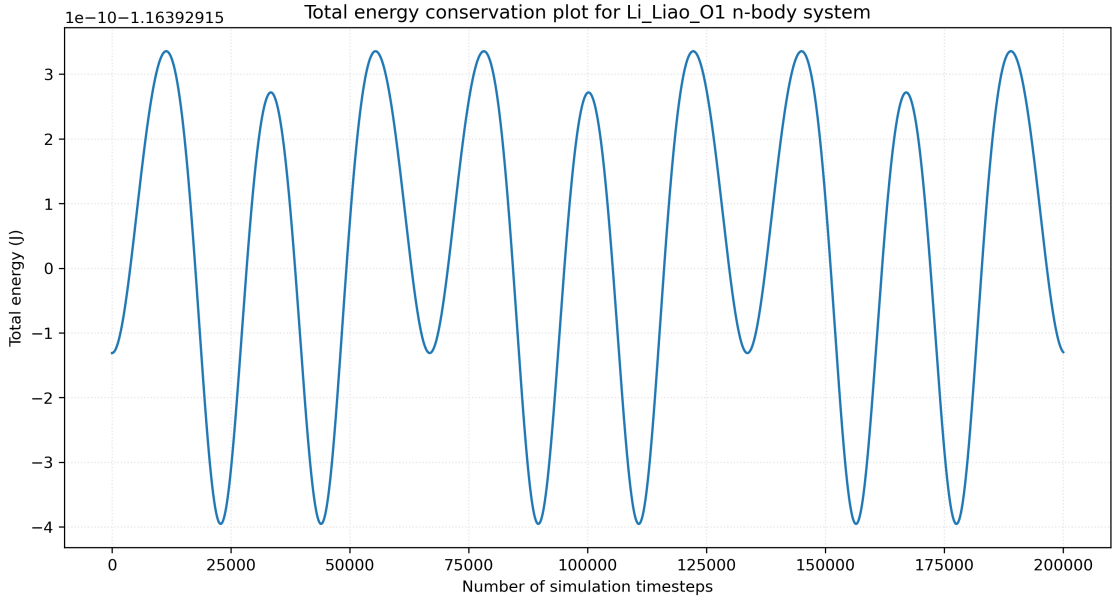

```
Median system total energy is -1.281977077834009 J
Median system angular momentum is 0.16828416049222947 kgm²/s
Period is 82.63810000000001(s)
```

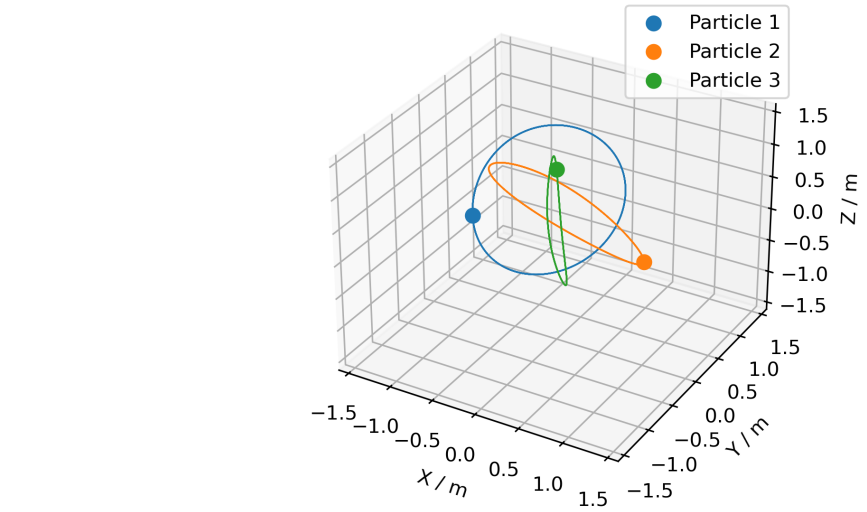
[27]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'choreography_O231 trajectories, nsteps=8263840,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(82.63810000000001))
```

[28]:
```
# Li Liao O1
run_integrator('Li_Liao_O1_ICS.txt')
```

```
Running integrator: 100%|      | 200000/200000 [00:06<00:00, 30001.83it/s]
Calculating angular momentum and total energy: 200000it [00:08, 23284.33it/s]
```


Total angular momentum conservation plot for Li_Liao_O1 n-body system
1e−12+2.094455511
Total angular momentum (kgm²/s)
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0
25000
50000
75000
100000
125000
150000
175000
200000
Simulation timestep (s)


Total energy conservation plot for Li_Liao_O1 n-body system
1e−10−1.16392915
Total energy (J)
3
2
1
0
−1
−2
−3
−4
0
25000
50000
75000
100000
125000
150000
175000
200000
Number of simulation timesteps

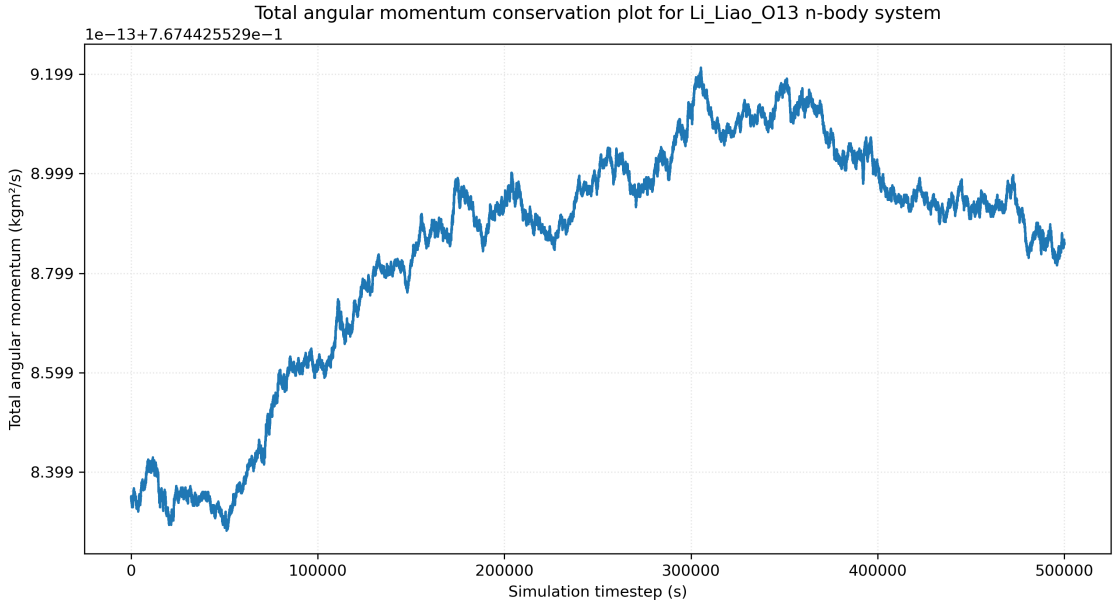


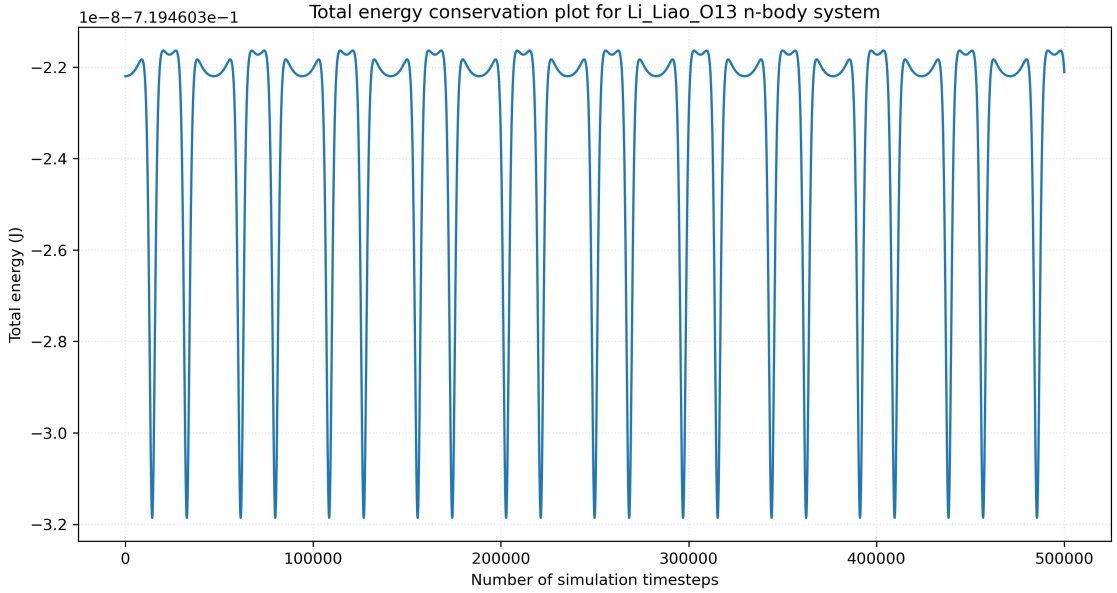

```
Median system total energy is -1.1639291499426911 J
Median system angular momentum is 2.0944555113826846 kgm²/s
Period is 6.678800000000001(s)
```

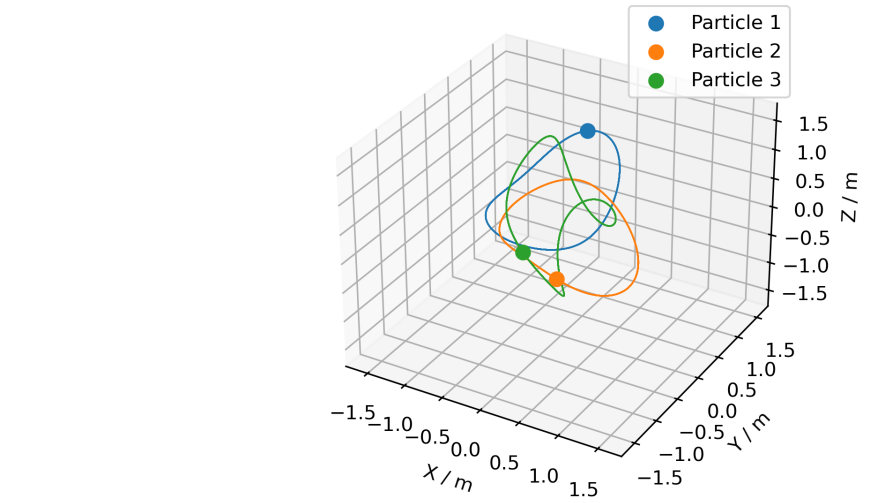
[28]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Li_Liao_O1 trajectories, nsteps=200000,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(6.678800000000001))
```

[29]:
```
# Li Liao O13
run_integrator('Li_Liao_O13_ICS.txt')
```

```
Running integrator: 100%|      | 500000/500000 [00:16<00:00, 30090.52it/s]
Calculating angular momentum and total energy: 500000it [00:21, 23411.00it/s]
```


Total angular momentum conservation plot for Li_Liao_O13 n-body system
1e−13+7.674425529e−1
9.199
8.999
8.799
8.599
8.399
Total angular momentum (kgm²/s)
0
100000
200000
300000
400000
500000
Simulation timestep (s)


Total energy conservation plot for Li_Liao_O13 n-body system
1e−8−7.194603e−1
−2.2
−2.4
−2.6
−2.8
−3.0
−3.2
Total energy (J)
0
100000
200000
300000
400000
500000
Number of simulation timesteps

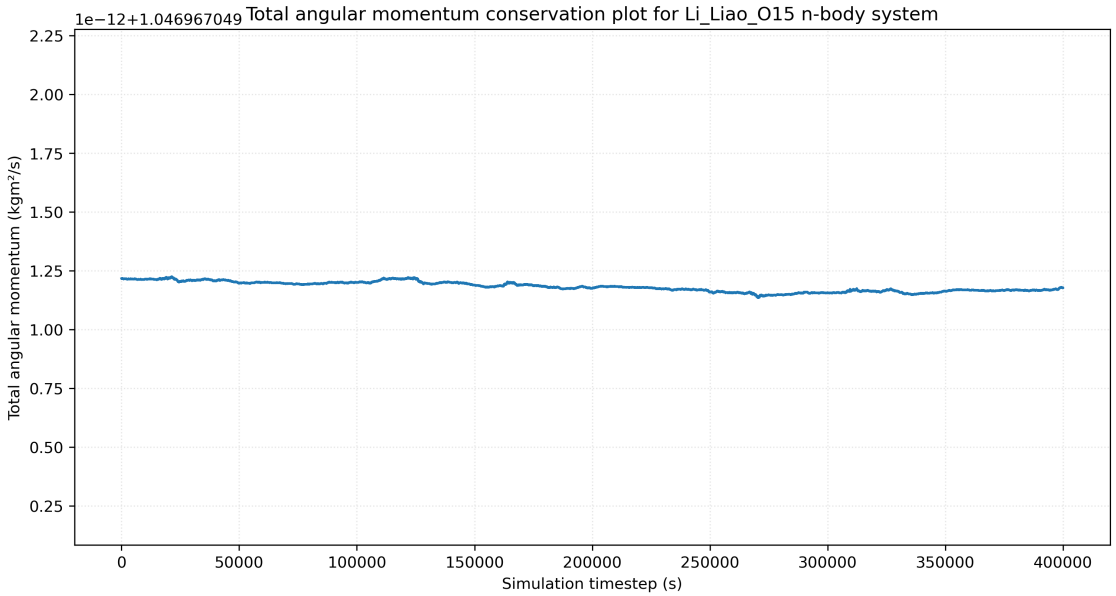


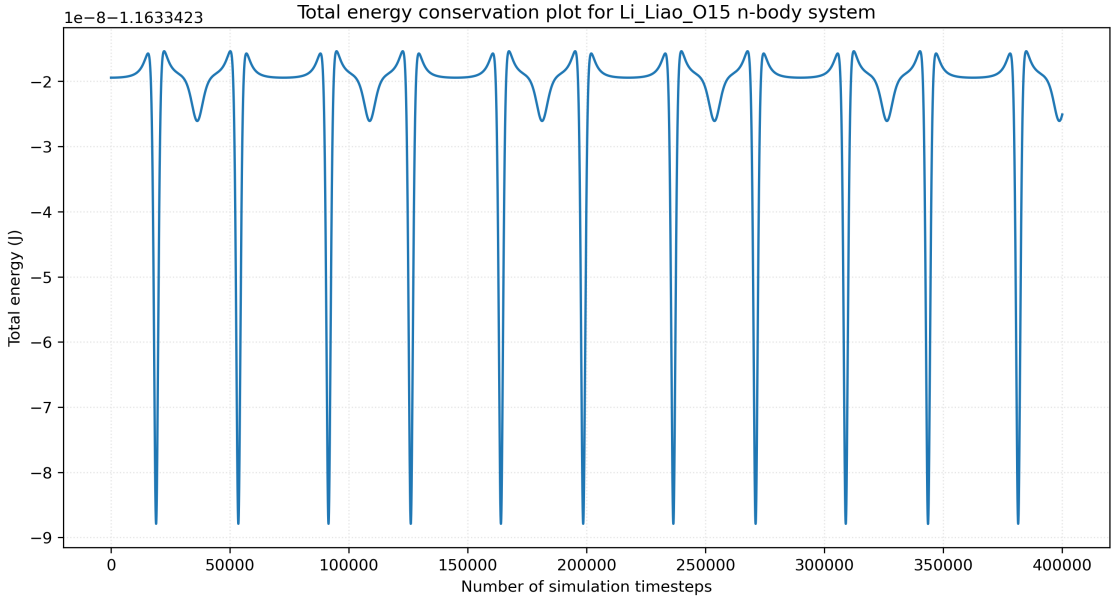

```
Median system total energy is -0.7194603221254359 J
Median system angular momentum is 0.7674425528728928 kgm²/s
Period is 9.4215(s)
```

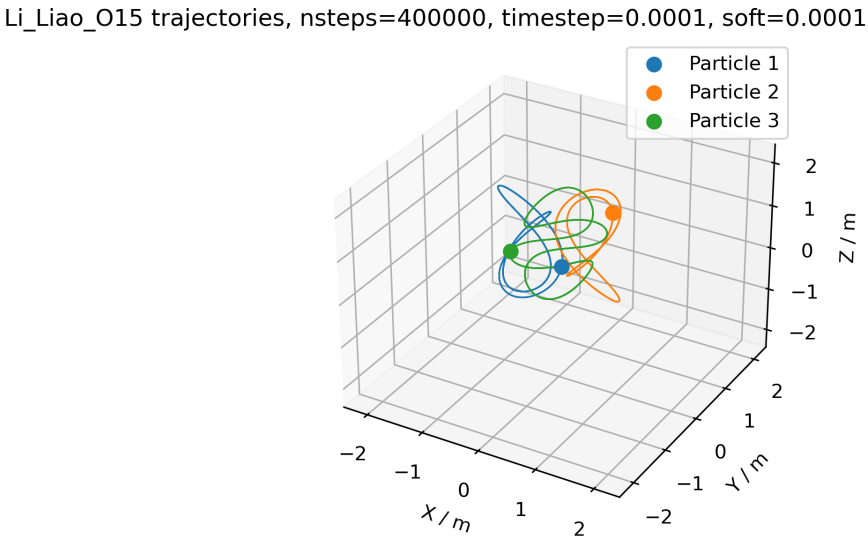
[29]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Li_Liao_O13 trajectories, nsteps=500000,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(9.4215))
```

[30]:
```
# Li Liao O15
run_integrator('Li_Liao_O15_ICS.txt')
```

```
Running integrator: 100%|      | 400000/400000 [00:13<00:00, 30184.23it/s]
Calculating angular momentum and total energy: 400000it [00:17, 23479.90it/s]
```


Total angular momentum conservation plot for Li_Liao_O15 n-body system
1e−12+1.046967049
Total angular momentum (kgm²/s)
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
50000
100000
150000
200000
250000
300000
350000
400000
Simulation timestep (s)


Total energy conservation plot for Li_Liao_O15 n-body system
1e−8−1.1633423
Total energy (J)
−2
−3
−4
−5
−6
−7
−8
−9
0
50000
100000
150000
200000
250000
300000
350000
400000
Number of simulation timesteps

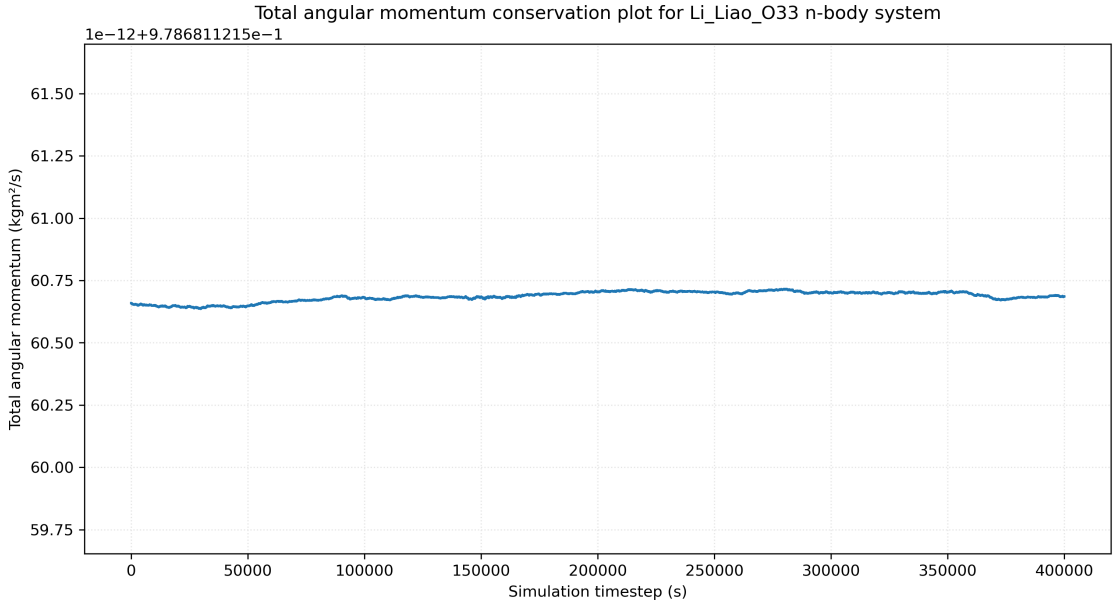


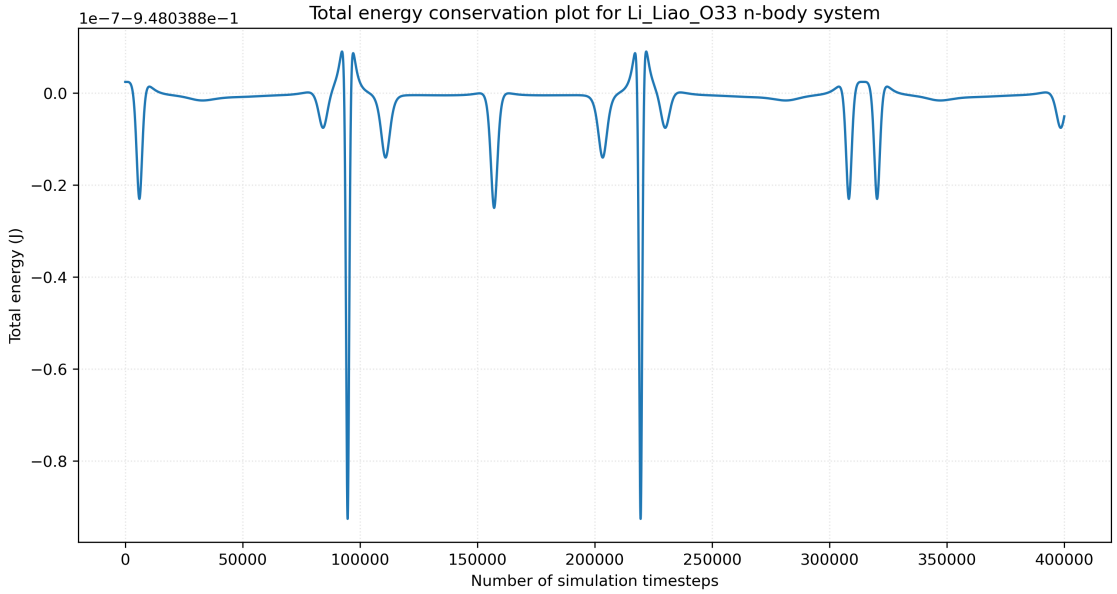

```
Median system total energy is -1.1633423192471255 J
Median system angular momentum is 1.0469670493711793 kgm²/s
Period is 14.499500000000001(s)
```

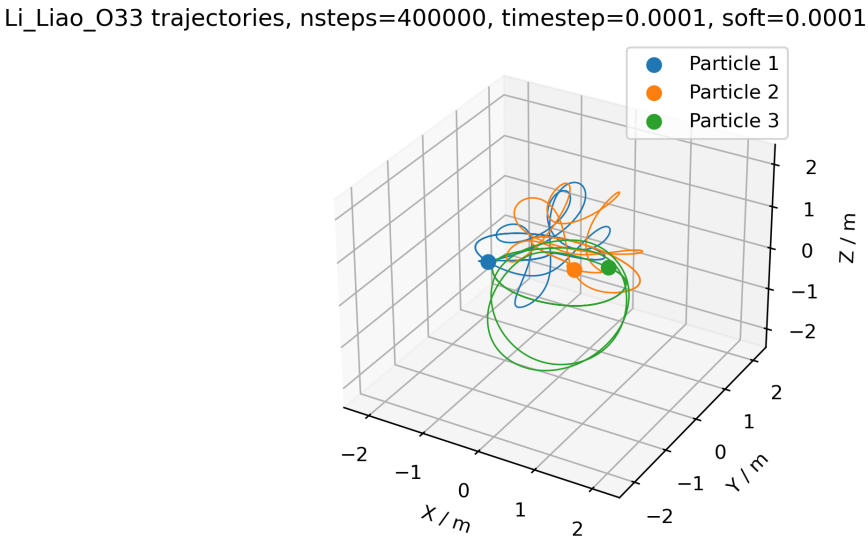
[30]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Li_Liao_O15 trajectories, nsteps=400000,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(14.499500000000001))
```

[31]:
```
# Li Liao O33
run_integrator('Li_Liao_O33_ICS.txt')
```

```
Running integrator: 100%|      | 400000/400000 [00:13<00:00, 30292.69it/s]
Calculating angular momentum and total energy: 400000it [00:17, 23459.66it/s]
```


Total angular momentum conservation plot for Li_Liao_O33 n-body system
1e−12+9.786811215e−1
61.50
61.25
61.00
60.75
60.50
60.25
60.00
59.75
Total angular momentum (kgm²/s)
0
50000
100000
150000
200000
250000
300000
350000
400000
Simulation timestep (s)


Total energy conservation plot for Li_Liao_O33 n-body system
1e−7−9.480388e−1
0.0
−0.2
−0.4
−0.6
−0.8
Total energy (J)
0
50000
100000
150000
200000
250000
300000
350000
400000
Number of simulation timesteps




```
Median system total energy is -0.9480388005868008 J
Median system angular momentum is 0.9786811215606888 kgm²/s
Period is 31.4236(s)
```

[31]:
```
(<Figure size 1200x1200 with 1 Axes>,
 <Axes3D: title={'center': 'Li_Liao_O33 trajectories, nsteps=400000,
timestep=0.0001, soft=0.0001, period_tol=0.0005'}, xlabel='X / m', ylabel='Y /
m', zlabel='Z / m'>,
 np.float64(31.4236))
```